Malicious Network Activity Detection using Deep Learning

Project Team

Muhammad Saad Hassan	P17-6137
Mubariz Ahmed Khan	P18-0010
Aamir Ahmad Khan	P17-6150

Session 2017-2021

Supervised by

Dr. Mohammad Nauman

Department of Computer Science

National University of Computer and Emerging Sciences
Peshawar, Pakistan

Dec, 2021

Student's Declaration

We declare that this project titled " *Malicious Network Activity Detection using Deep Learning*", submitted as requirement for the award of degree of Bachelors in Computer Science, does not contain any material previously submitted for a degree in any university; and that to the best of our knowledge, it does not contain any materials previously published or written by another person except where due reference is made in the text.

We understand that the management of Department of Computer Science, National University of Computer and Emerging Sciences, has a zero tolerance policy towards plagiarism. Therefore, We, as authors of the above-mentioned thesis, solemnly declare that no portion of our thesis has been plagiarized and any material used in the thesis from other sources is properly referenced.

We further understand that if we are found guilty of any form of plagiarism in the thesis work even after graduation, the University reserves the right to revoke our BS degree.

Muhammad Saad Hassan Signature: _____

Mubariz Ahmed Khan Signature: _____

Aamir Ahmad Khan Signature: _____

_____

Verified by Plagiarism Cell Officer

Dated:

Certificate of Approval

The Department of Computer Science, National University of Computer and Emerging Sciences, accepts this thesis titled *Malicious Network Activity Detection using Deep Learning*, submitted by Muhammad Saad Hassan (P17-6137), Mubariz Ahmed Khan (P18-0010), and Aamir Ahmad Khan (P17-6150), in its current form, and it is satisfying the dissertation requirements for the award of Bachelors Degree in Computer Science.

Supervisor

Dr. Mohammad Nauman Signature: _____

_____

Mashal khan

FYP Coordinator
National University of Computer and Emerging Sciences, Peshawar

_____

Dr. Hafeez ur Rehman

HoD of Department of Computer Science
National University of Computer and Emerging Sciences

Acknowledgements

First of all, we are thankful to the Almighty that He bestowed His generous blessings upon us in a shape of sound health and well-being which were the primary touchstone for the accomplishment of this project.We are also thankful to our parents for their support and attention that we needed. They encouraged us at each stage of life and firmly supported us through our thin and thick, for which we are highly indebted to them.We acknowledge our deepest and profound gratitude to our worthy and valuable supervisor, Dr. Mohammad Nauman for introducing and igniting the love of computer science in our hearts, We are extremely thankful to him. We will always be indebted to him for sharing his expertise, sincere and valuable guidance and continuous encouragement which enabled us to pursue and complete this project and whole heartedly pursue computer science as a career. We would also like to express our deepest gratitude to Ms. Mashal Khan and Dr. Zeeshan Khan Alvi for their guidance, intellectual suggestions and support, We would also like to extend our deepest gratitude to all the Department faculty members for their help and support.

Muhammad Saad Hassan

Mubariz Ahmed Khan

Aamir Ahmad Khan

Abstract

The current modern era is known as the era of data. Let's look at the word data; the word data means information, this information can be knowledge about anything. If we want to look at a particular event, we can summarize that certain event if we have data , If we have data about a certain event, we can analyse that certain event to such an extent that whenever that certain event occurs we perform an action to the response of that event. Man is a creature which is known to change the lives of mankind for the better, Hence using data, information or knowledge, we must try to better the lives of others around us.

Since in the modern era, the importance of data is no secret, hence it increases the value of data, Since we all users of the internet, we must learn to protect our data from the one's with malicious intents. Huge multi-billion dollar organizations such as Amazon, Facebook, Microsoft, Tesla all operate using modern day computer networks, they all have measures to protect their data from anyone that can harm them, we as users of the internet must also take precautionary measures to protect ourselves.

With the outbreak of the Corona Virus, Work Places, Businesses and Institutions around the globe have shifted to the idea of Working from Home, which has enabled people around the world to stay connected via the Internet. The rise of data being shared across the internet has further given Cyber-Criminals all the more reason to Attack and Intercept the shared information. Due to increased public awareness about the importance of keeping online transactions and documents secure, Cyber-Criminals have also modified their network attack strategies making it extremely difficult for old-school intrusion detection systems or network anomaly detection systems to effectively analyze and identify abnormal traffic.

Since we have data of malicious streams of several types, we can analyse and process those malicious streams to learn and understand the nature and patterns to classify those streams, hence we propose a Deep Learning based solution, which analyses Network

Activity sequences to classify whether the certain node is Malicious or Benign. This project is about the mechanism we have constructed which will capture real-time network stream, and pass it through a deep neural network to inform the system administrator if a certain node in a network is producing malicious stream.

Contents

1 Introduction **1**

1.1 Introduction 1

1.2 Motivation 2

1.3 Objectives 3

1.4 Project Scope 3

2 Review of Literature **5**

2.1 Dataset Literature Review 5

2.1.1 ISCX-IDS CIC 2012 Dataset 6

2.1.2 ISCX-IDS CIC 2017 Dataset 7

2.1.3 Communications Security Establishment-Canadian Institute for Cybersecurity-IDS2018 on AWS 8

2.1.4 Attacks Performed 10

2.2 Models 12

2.2.1 Decision Tree Classifier 12

2.2.2 Random Forest 13

2.2.3 ExtraTree 15

2.2.4 BERT 17

2.2.4.1 Pre-Training: 20

2.2.4.2 Fine-tuning: 21

2.2.4.3 Summary 22

2.2.4.4 Conclusion 22

2.2.5 Multi-Layer Perceptron 23

2.2.6 AutoEncoder 23

2.2.7 Variational Autoencoder 24

2.3 Related Works and Background 25
2.3.1 Malware Analysis 25
2.3.2 Static Malware Analysis Vs Dynamic Malware Analysis 26
2.3.3 Deep Neural Solutions 26

3 Unified Modelling Language **29**
3.1 Use Case Scenario 29
3.2 Use Case Diagram 29
3.3 Activity Diagram 31
3.4 Class Diagram 32

4 Methodology **35**
4.1 Model Preparation 35
4.1.1 Data Collection 35
4.1.1.1 Data Parsing 35
4.1.1.2 Excerpt of Parser Class 36
4.1.2 Data Preprocessing 37
4.1.2.1 Excerpt of Preprocessor Class 37
4.1.3 Feature Engineering 38
4.1.4 Experiment's Setup 38
4.1.5 Experiment's Performed 39
4.1.6 Dataset Analysis 39
4.1.6.1 Fin Flag 40
4.1.6.2 Synchronize Flag 41
4.1.6.3 Acknowledgment Flag 42
4.1.6.4 Push Flag 43
4.1.6.5 CWE Flag 44
4.1.6.6 Urgent Flag 45
4.1.6.7 Number of packets transferred per second 46
4.1.6.8 Destination Ports Attacked 47
4.1.6.9 Flow duration of each sort of flow 48
4.1.6.10 Dataset Construction 49

4.1.6.11 Dealing with Highly Imbalanced Data 49
4.1.6.12 Models 50
4.2 Workflow Sequence 51
4.2.1 Automated Data Collection 51
4.2.1.1 Dumpcap 51
4.2.1.2 What is PCAP File Format? 53
4.2.1.3 Splitcap 54
4.2.2 Automated Conversion 55
4.2.2.1 PRC Flowmeter 55
4.2.3 Automated Feature Scaling 56
4.2.3.1 Automated Preprocessing 56
4.2.3.2 Automated Normalization and Standardization 57
4.2.4 Automated Model Prediction 58
4.2.5 Maintain Activity log file 58

5 Results **59**
5.1 Results 59
5.1.1 Evaluation Metrics 59
5.1.2 Precision: 59
5.1.3 Recall: 59
5.1.4 F1 Score: 59
5.1.5 Datasets 60
5.1.6 Dataset A 60
5.1.6.1 Results on Dataset A 60
5.1.7 Dataset B 61
5.1.7.1 Results on Dataset B 62

6 Discussion **63**
6.1 Work Breakdown for FYP-1 64
6.2 Work Breakdown for FYP-2 65

7 Conclusions and Future Work **67**

A Data Preprocessing **69**

A.1 Parser Class 69

A.2 Preprocessor Class 72

B Models **79**

B.1 Muti-Layer Perceptron 79

B.2 AutoEncoder Model 80

References **85**

List of Figures

2.1 Testbed Architecture 7
2.2 DDOS Attack 10
2.3 Portscan Attack 11
2.4 Decision Tree. 13
2.5 Random Forest 14
2.6 ExtraTree 16
2.7 LSTMs Modal - Long Short Term Memory 17
2.8 The Transformer - model architecture. 19
2.9 Overall pre-training and fine-tuning procedures for BERT. 20
2.10 BERT input representation. 21
2.11 Fine-tuning BERT 21
2.12 Multi-Layer Perceptron 23
2.13 AutoEncoder 24
2.14 Variational AutoEncoder 25
2.15 Traditional vs. advanced malwares. 26

3.1 Use Case Diagram of Malicious Network Activity Detection 30
3.2 Activity Diagram of Malicious Network Activity Detection 31
3.3 Class Diagram of Malicious Network Activity Detection 32

4.1 Acknowledgement flag values to each type of stream 40
4.2 Synchronize flag values to each type of stream 41
4.3 Density Plot of Rest Flag to each type of stream 42
4.4 Density Plot of Push Flag to each type of Network Flow 43
4.5 Density Plot of Congestion Warning Flag to each type of Network Flow . 44
4.6 Density Plot of Congestion Warning Flag to each type of stream 45

4.7 Distribution of flow duration to each stream 46
4.8 Distribution of flow duration to each type of stream 47
4.9 Distribution of flow duration to stream 48
4.10 Acknowledgement flag values to each type of Network Flow 49
4.11 Work Flow of Malicious Network Activity Detection 51
4.12 Dumpcap illustration 52
4.13 raw pcap file containing network packets from which we are extracting our dataset. 53
4.14 Splitcap illustration 55

List of Tables

2.1 GLUE Test results 22

5.1 Results that we achieved by training on dataset A 61

5.2 Results that we achieved by training on Dataset B 62

Chapter 1

Introduction

This chapter introduces what the project is all about, the motivation behind doing this project, the objectives and scope of the project.

1.1 Introduction

Computer Network Systems have been plagued with malicious network activity since the inception of Computer Networks, and as the modern day era is the most rapidly changing and evolving era of modern technology, Attackers have also evolved their attacking strategies. Rule based analysis methods and Static and Dynamic Analysis were used to detect attacks but are proven to be incompetent to compete with modern day malicious applications.

Modern day research has implored the use of Deep Learning to solve modern day security problems, Deep learning constructs artificial neural networks and is built to accommodate large volumes of unstructured data. Deep Learning is a sub-branch of Machine Learning which is a sub-branch of Artificial Intelligence; what it really does is imitate the way the Human Brain behaves by passing information on one layer of neurons to the other and thus helps in training a model based on provided feedback.

As, deep learning systems are being used to provide intelligent solutions in man's life; the aim of this project is to devise a tool or a mechanism which would be able to detect if a malicious node is connected to the network on the basis of passing real-time-stream network data through an integrated state-of-the-art neural architecture. The integrated neural architecture will inform the system administrator about the malicious ip via dumping logs in a log-file.

1.2 Motivation

Many global organisations have turned their attention to the Importance of Cyber-Security. Many Research Organisations have also demonstrated success in Malicious Network Activity Detection and Malware Detection on the basis of Deep Learning Models. Malware Detection on Highly Imbalanced Data through Sequence Modeling uses BERT and LSTM Network to learn Android Malware API Call Sequences, and have been successful in achieving high accuracy scores thus proving Neural Language Models can be used to achieve high score in various Security tasks.

The research of Radfordet al. [18] introduces a model trained on computer network traffic to detect and classify anomalies.The work of Hanet al.[8] proposes an attention model which processes computer network packet streams with an adjustable length to detect payload-based attacks. The works of P. Prasse [25] deals with the problem of detecting malware on the basis of encrypted https traffic analysis on the basis host address,timestamps and data volume information of aggregated packets. The neural language model used is LSTM. As with each passing day, Cyber Criminals and constructing new and improved methodologies to manipulate a Network, New and Advanced Techniques must be used to secure a Network, and with the success of Neural Language Models on Malware Detection, gives us all the more reasons to use Deep Neural Architechtures for the classification of Network Activity.

1.3 Objectives

The objectives for our project 'Malicious Network Activity Detection using Deep Learning' are:

- Data Collection for training model.
- Data Preprocessing for data to be in suitable format for our model.
- Feature Engineering and Feature Extraction.
- Training Deep Neural Model.
- Selection and Training of a new Deep Learning Models
- Construct a automated mechanism; that will capture "real network stream data"
- The captured real time stream network data is to be preprocessed and standardized via automation; so that is ready for the deep neural network.
- Processing "captured real network stream data" and test whether it is malignant or benign by passing it through or model
- Notifying administrator if malicious activity detected via dumping logs into log files.

1.4 Project Scope

The scope of the project is that, a central node with a Deep Neural Model deployed on it, will be connected to a certain network. The node will be the central node or the administrator as the administrator node will be the one which will be attacked by one's with malicious intent. The node will capture real-time-stream-data as input, process and parse the raw real-time-stream-data such that it is suitable to be fed to our model. As a Trained Model will be deployed on the node, The model will be able to classify whether the activity in the Network is Malicious or not. If the activity is found to be Malicious, The Administrator will be notified to the mitigate the certain Malicious Node.

Chapter 2

Review of Literature

This chapter provides a comprehensive literature review of the related work to this project, the different models we performed experiments on, the methodology and our proposed system for Malicious Network Activity Detection using Deep Learning.

2.1 Dataset Literature Review

As the modern era is progressing, and since the arrival of 5g, this has automatically given birth to a huge increase in number of inter-connected devices; that increases all the more reason for attackers to manipulate systems. Anomaly detection schemes and Intrusion Detection Systems has proven to be a vital defense mechanism; but a huge number of modern day produced datasets lack the information needed for a modern day defense mechanism. There are several modern day datasets(such as DARPA98, KDD99, ISC2012, and ADFA13) which exist but lack the required features which would make them suitable in the current time. A huge number of these datasets are out-dates, a variety of these datasets don't seem to resemble modern day network streams, or the don't contain feature set and meta data[16].

2.1.1 ISCX-IDS CIC 2012 Dataset

Anomaly-based techniques to network intrusion detection (IDS) in particular suffer from inaccuracy in evaluation, comparison, and deployment due to a lack of suitable datasets. Many of these datasets are internal and can't be released because of privacy concerns; the rest are greatly anonymous and lack resemblance with modern day network streams; and yet others lack important statistical qualities. These drawbacks are the main reasons why a flawkess dataset has yet to be created. As a result, researchers are forced to rely on datasets that are often suboptimal.

ISCX2012 has been divided into two profiles, An Attacker-Profile and a Benign-Profile, using which multi-stage attack scenarios are carried out. The dataset contains the network traffic for Simple-Media-Transfer-Protocol (SMTP), Secure-Shell-Protocol (SSH), Internet-Message-Access-Protocol (IMAP), Post-Office-Protocol 3 (POP3) and File-Transfer-Protocol(FTP) with full packet payload. However, there are no traces of HTTPS in this dataset, which consists of 70-percent of today's modern network traffic. [2]

2.1.2 ISCX-IDS CIC 2017 Dataset

Two Networks were designed and implemented to create a vast data-set. An Attack Network and a Victim Network is a network which is secure with switches, firewalls and routers installed. The victim network has the most commonly used operating system's installed on it, where as the attack-network is a total different infrastructure with a collection of personal machines with public ip's and necessary tools and applications installed to perform each type of Attack(Distributed Denial of Services, Port-scan etc.), The attack network has kali linux and windows 8.1 installed on it to perform each attack scenario. [4]

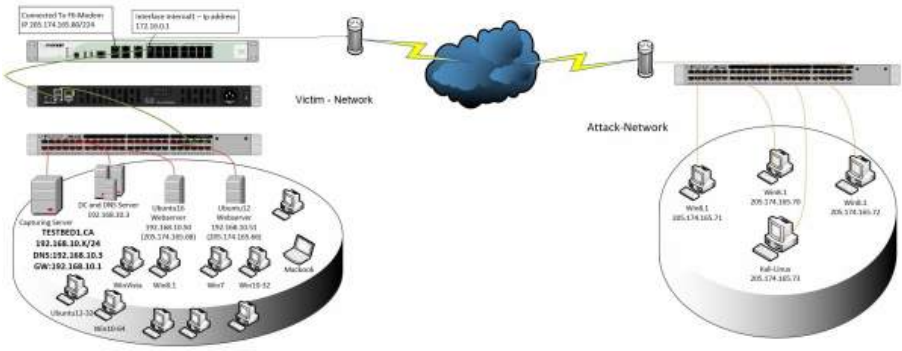


Figure 2.1: Testbed Architecture

2.1.3 Communications Security Establishment-Canadian Institute for Cybersecurity-IDS2018 on AWS

Many academics have focused on anomaly detection because of its potential for identifying novel attacks. However, due to system complexity, which necessitates extensive testing, assessment, and tuning prior to deployment, its applicability to real-world applications has been impeded. The best way to gauge the competence of these systems is to run these systems on real labelled network streams; and those network streams containing malicious streams which mimic the real life nature of malicious streams.

This is a significant challenge in and of itself, as datasets such as those are scarce as sharing them will raise privacy concerns and the one's that exist contain data which anonymized, hence this results in researchers using datasets which are simple inadequate. As modern day networks and the type of streams are evolving, it is essential to move towards dynamically generated datasets that not only represent modern day traffic streams and attacks but also that can be changed, reproduced and also extended.

A well planned, thought-out mechanism is needed to produce datasets for analysing, testing, and evaluating anomaly based network intrusion systems. the primary concern for construction of this mechanism was to generate a state of the art dataset, with it's own characterstics covering variety of features which were previously not measured. [3]

This dataset contains 7 types of attacks such as:

1.Brute-force

2.Heartbleed

3.Botnet

4.DoS

5.DDoS

6.Web attacks

7.Infiltration Attacks

The Attacker Network has a total of 50 personal machines; where as the victim network has a collection of 5 departments containing a total of 30 servers with 400 workstations connected to it. The streams generated by this system were captured using CICFlowMeter-V3 and contains 80 extracted features. This dataset has proven to be one of the best datasets for network oriented intrusion detection systems.[18]

2.1.4 Attacks Performed

- **Brute-Force-Attack:** These are attacks are one of the most commonly performed attacks, they're commonly used for cracking passwords; Brute Force attack is basically trying all possible options basically exhausting the search space and can also used to find hidden information in an application.

- **Heart-Bleed-Attacks:** Heart-Bleed-Attacks come from a bug in OpenSSL cryptography library, That bug is used by Attackers by sending a heartbeat request which is distorted and contains a miniature pay load, with a huge length; this at time provokes a server to respond.

- **Botnet:** A botnet owner can use a number of Internet-connected devices to perform tasks such as data-theft, sending spam data and can also the Attack to access the device and connection.

- **DoS Attack:** The attacker floods the victim's machine with many repeated requests in order to overload the system so that valid requests don't reach the victim's machine.

- **DDoS Attack:** DDoS Attack or Distributed Denial of Service happens when the attacker uses multiple machines or systems to repeatedly send multiple requests to overload the victim's network so that valid requests aren't recieved by the victim. A botnet flooding a certain system is an example of a DDoS attack.

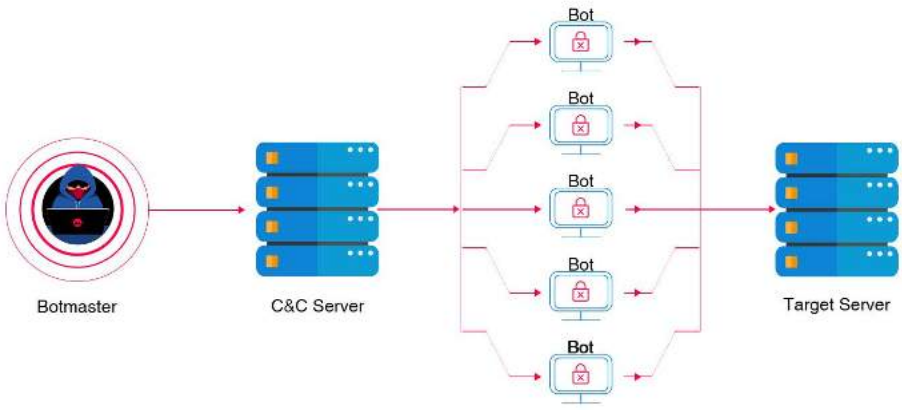


Figure 2.2: DDOS Attack

- **Web Attack:** Web Attacks such as SQL injection, Cross-Site Scripting attacks are also one of the most common modern day attacks.Sql Attack occurs when the attacker tries to force information out of a database by querying the database with a string of SQL commands. Cross-Site Scripting occurs when attackers take advantage of the lack of testing of the code by the developer. A developer must eliminate the possibilty of SQL injection and Brute Force attacks.

- **Infiltration Attack:** Infilteration attack is carried out when the attacker manipulates the victim to execute a compromised software; now as the malicious code is now on the victim's network, it can perform heavy damage such as scan every port, service enumerations using nmap.

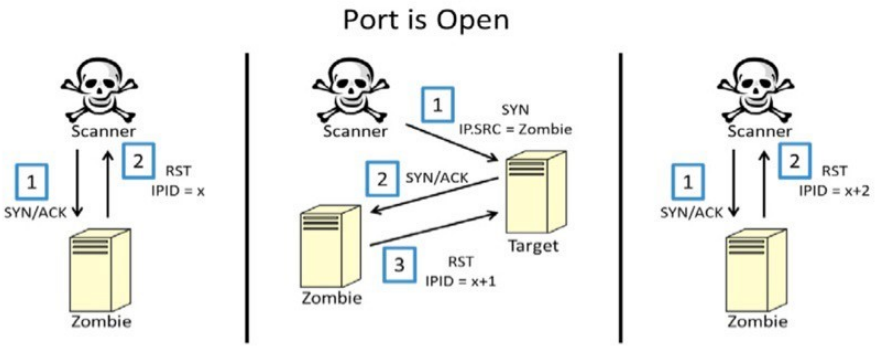


Figure 2.3: Portscan Attack

2.2 Models

2.2.1 Decision Tree Classifier

Decision Tree belongs to the category of supervised Machine Learning technique, Ensemble methods of the base decision tree has also given birth to other very useful models such as RandomForest and extremely randomized trees which can be used for feature extraction. They can also be used for regression tasks. A decision tree is basically a tree based structure, where the nodes represent the features, each branch of the tree leads to the decision that is being made. What makes this whole tree work is the beauty of Entropy. They're easy to construct and have a small inference time.

Important Terminology related to Decision Trees:

- **Root Node:** The root node is the feature with the highest importance in the tree, as this feature will be the node which splits the data the best. It's the parent node of the of the whole tree; Variance is used for computing the homogeneity of a node.
- **Splitting:** When a node is divided into several nodes; it is called splitting of a tree.
- **Decision Node:** When a node's sub-node ha more information gain so it split's into further sub nodes.
- **Leaf Node:** Node that do not further divide into sub-nodes is leaf or terminal node.
- **Pruning:** Its data compression technique, section of decision tree that is non-critical and redundant to classify instances.
- **Sub-Tree:** The subsection of the whole tree is call sub tree.
- **Child and Parent Node:** A node which further splits into sub-nodes is parent node and sub-nodes are child nodes.

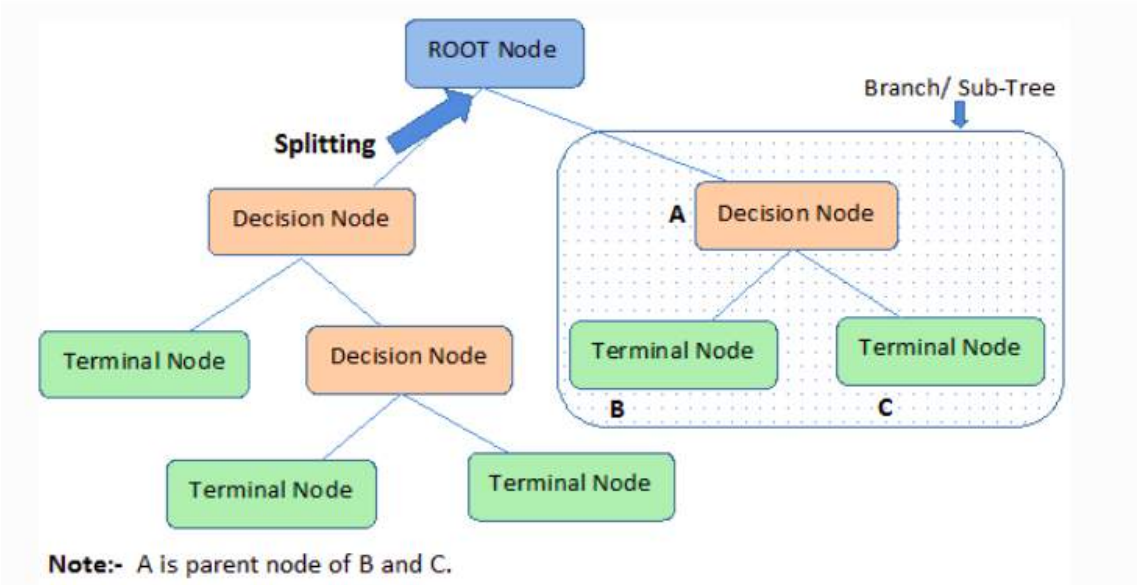


Figure 2.4: Decision Tree.

To test if splitting is "Best" in individual classes, a decision tree method can be used to see if the splitting criteria produces a node with only one sort of target value attributes. We can also call the splitting branch as pure, if it produces only sort of feature target value. [15]

2.2.2 Random Forest

Random Forest is an ensemble learning method based on the decision tree classifier, it is basically a collection of tree's; where each tree is constructed by values of a random vector which is sampled independently, and for all trees in the forest; it'll be the same distribution.The generalization error will converge to a limit as the amount of trees get larger. The number of trees that a forest contains and their correlation determine the generalisation error of the forest. Randomly selecting features to produce a split at the current node will produce error rates similar to Adaboost, but compared to Adaboost will be more robust to noise. Internal estimates produce a response to the growing number of features used in splitting, they also estimate track error strength, correlation. The importance of variables is also assessed by internal estimates.

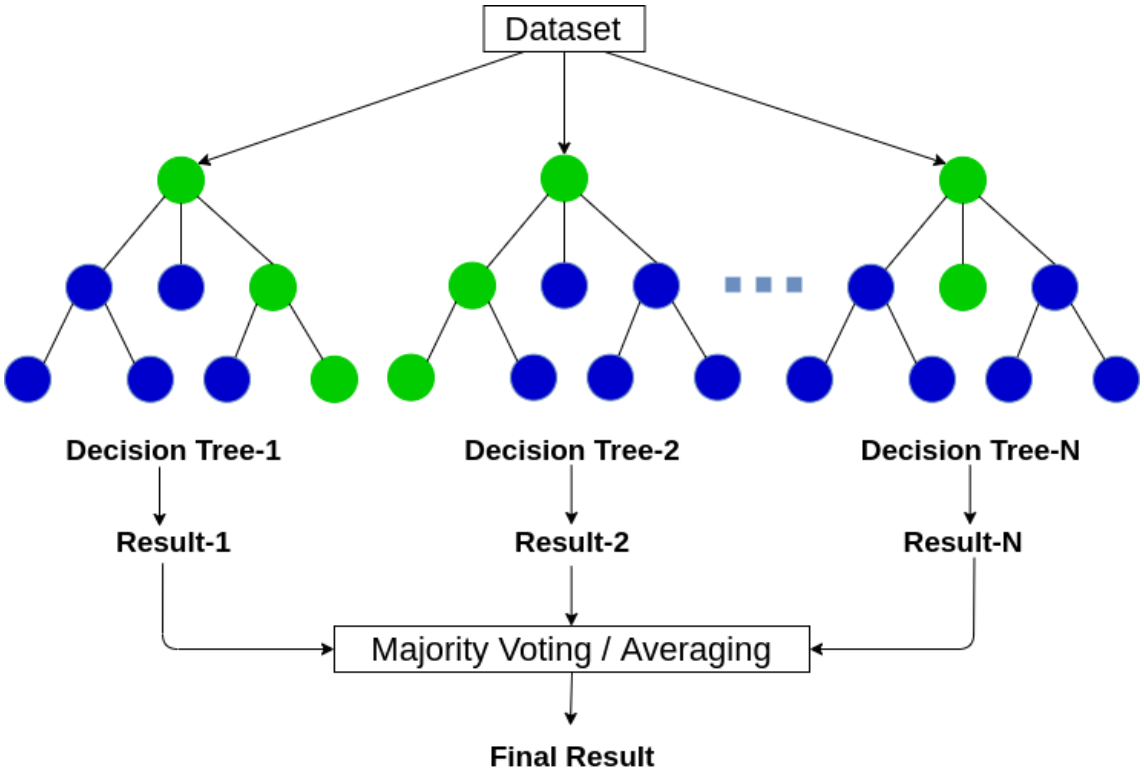


Figure 2.5: Random Forest

Random Forest; as mentioned before is an ensemble learning method, which is based on multiple decision trees which work together, each single tree will produce a class prediction; hard voting will be performed; and thus class label value will be predicted. What makes the random forest work so well is that it contains largely uncorrelated trees which all concurrently work together to outperform an individual tree. Miniature changings in the training set will construct an altogether completely different tree. Random Forest employs the bagging approach by enabling each distinct tree to sample from the current dataset at random replace, giving us diverse trees.

2.2.3 ExtraTree

ExtraTrees or Extremely Randomised Trees is fundamentally a decision tree, ExtraTree is ensemble learning method just like RandomForest, it minimizes generalisation of data by randomising decisons and constructs subsets of data. Let's compare the base decision tree model with it's ensemble methods in order of variance ranging high to low, ending in ExtraTreesClassifier.

Decision Tree(High Variance)

Decision Tree, the base classifier is more prone to generalise the data since it learns from only one pathway of decisions, a single decision tree frequently overfits the data in it's truest form. Predictions based on a single decision tree are rarely appropriate when applied to new information.

Random Forest (Medium Variance)

By injecting randomization, random forest models lower the danger of overfitting by:

- generating numerous trees (n estimators)
- drawing observations with replacement (i.e., a bootstrapped sample)
- dividing nodes based on the optimal split among a random fraction of the features collected at each node

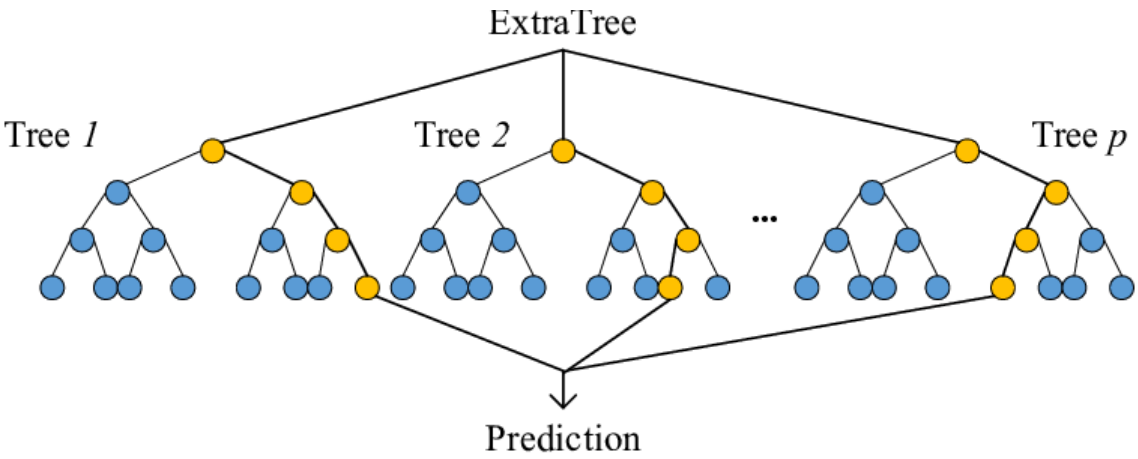


Figure 2.6: ExtraTree

Extra Trees (Low Variance)

Extra Trees is similar to Random Forest in that it generates several trees and splits nodes using random subsets of features, but it differs in two major ways: it does not bootstrap observations (meaning it samples without replacement) and nodes are split on random splits rather than optimal splits. In summary, ExtraTrees:

- constructs many trees by default with bootstrap = False, which implies it samples without replacement
- Nodes are split based on random splits among a randomly selected subset of features at each node.

Randomness in Extra Trees originates from the random splits of all observations, rather than from data bootstrapping.

2.2.4 BERT

LSTMs

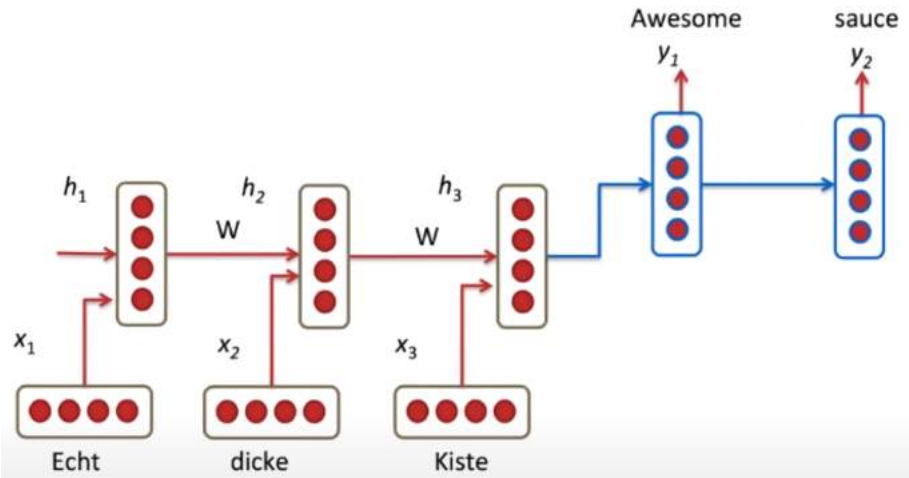


Figure 2.7: LSTMs Modal - Long Short Term Memory

The Neural Network architechture of the transformer was initially invented to put a rest to the problem of translation speech from one language to another. The transformer architechture was much appreciated as before them Long short term memory or LSTM neural networks were used to solve this task, but they issues of their own. They are extremely time extensive and text is processed and generated sequentially; Long short term neural networks aren't really great at understanding the context in a body of text and Bi-directional Long Short Term Memory neural networks also come up short.Bi-directional lstm's aren't really great as well as they learn the context left to right, right to left; and then concatenates them to understand the context; hence losing the meaning behind the context of that certain text.

Transformers

The transformer architecture has proven to be a revelation in modern day language processing; the transformer model architecture answers questions which long short term memory networks failed to answer. The transformer architecture consists of two parts; an encoder and a decoder. Let's say we want to translate English to Japanese; we'll pass the english words to the encoder model and the encoder will generate embeddings for every word in that body of text. The decoder will accept the embeddings generated by the encoder and will generate translation English to Japanese. [9]

LSTMs Vs Transformers

What makes this seperation of tasks so conceptually appealing than long short term memory networks? This seperation of tasks enables the encoder to learn and contextually understand the english language much better than lstm networks; where as the decoder part learns how does one language map onto the other; as of now, pre-trained transformer models are available in a variety of languages. We can construct multiple deep neural language architechtures using transformer models. Stacking Encoders gives us BERT; Stacking Decoders will give us the GPT-3 model.

A transformer is a deep neural language model which uses attention, which is calculating the significance of every part of the input data; It was introduced into the field of NLP or natural language processing but as of now, we can see transformer based models used in the field of computer vision. BERT stands for Bidirectional Encoder Representations from Transformers, is a representation of stacking encoder models on top of each other.BERT was created and published in 2018 by Google. We can use BERT to learn:

- Neural Machine Translation
- Question Answering
- Sentiment Analysis
- Text summarization

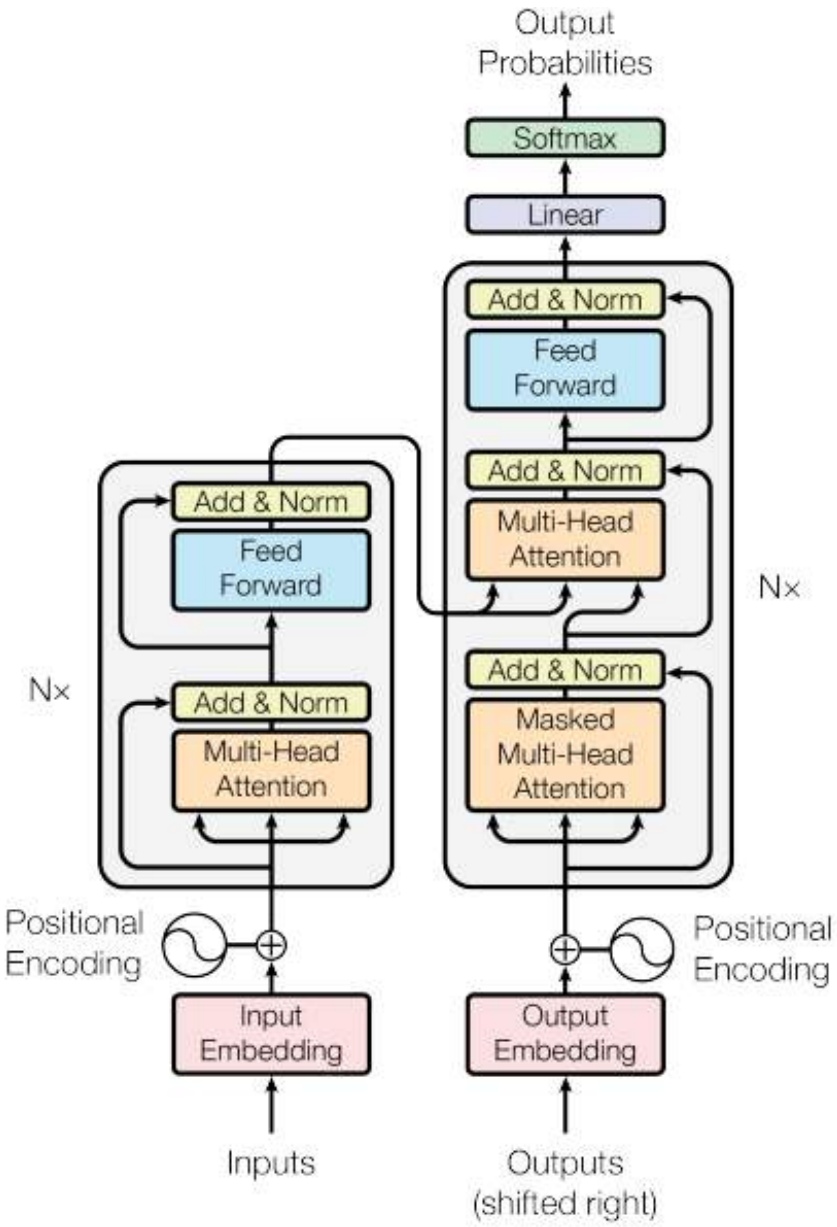


Figure 2.8: The Transformer - model architecture.

To solve problems of the modern day natural language, in order to perform well, the deep neural language model must be able to understand the natural language. So, we can train BERT to comprehend natural language, and then fine-tune the classification layers for our downstream task. For the pre-training and the fine-tuning part the neural architectures will be identical; the concept of transfer learning is essential here, as different models can be used for different tasks just by fine-tuning the output layer. All of the parameters are trained in the output layer; it contains two special tokens [SEP] and [CLS].

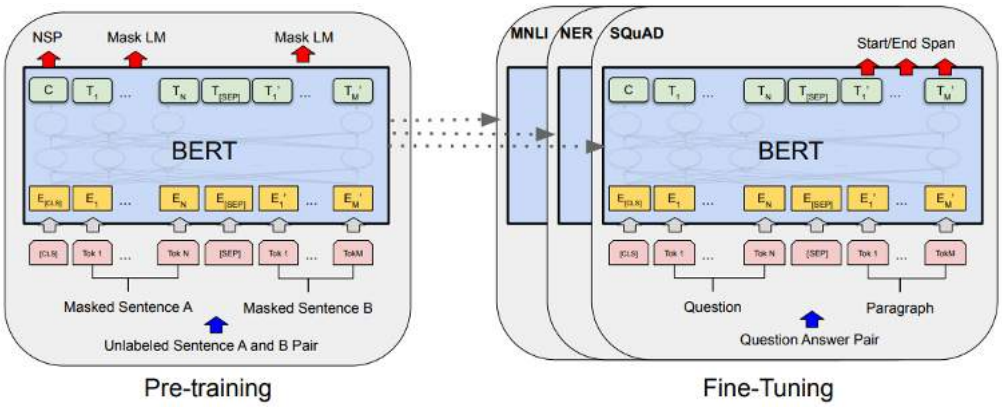


Figure 2.9: Overall pre-training and fine-tuning procedures for BERT.

2.2.4.1 Pre-Training:

The main outcome of pre-training the model is to make the model understand context and to make it understand what language really is. BERT learns language by training on two unsupervised methods; Masked-language-modelling and next-sentence-prediction. What masked language modelling really is training the layers with masking a word in a sentence and making the model predict the masked word; making the model really understand how a word fits in a language. Next Sentence Prediction is as the name suggest what it is; predicting which sentence occurs after a sentence.

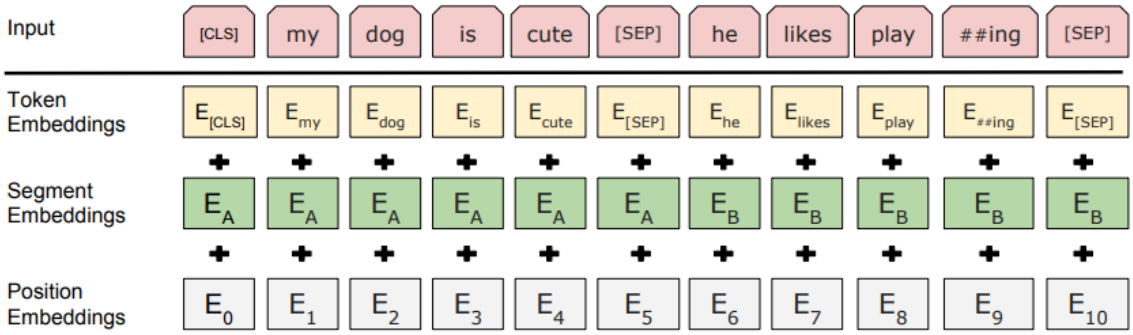


Figure 2.10: BERT input representation.

The bert tokenizer will tokenize each and every word according to it's pretrained layers. Each flavour of Bert will have different weights of it's pretrained layers.Summing up these token embeddings, segmentation embeddings and position embeddings gives us the input embeddings; As all of these vectors are fed to bert all at once, we need the segement and position embeddings for ordering. Our model BERT also needs this ordering to be preserved.

2.2.4.2 Fine-tuning:

When we obtain a pretrained BERT model, now we can fine-tune bert's classification layers to perform our specific task such as text summarization, text classification, question answering. If we take the example of question and answering, all we need to do is fine-tune our final classification layers on the question answering dataset; we can perform such a task on any natural language task. As we only have to fine tune only one or few classification layers, this won't be time and computationally extensive. [13] (see Figure 2.5)

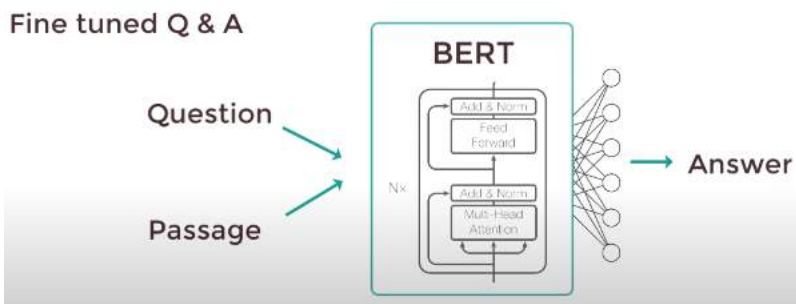


Figure 2.11: Fine-tuning BERT

2.2.4.3 Summary

The BERT SQuAD; which is a Q and A model constructed by Stanford University, takes only 30 minutes to fine tune and gives a 91 percent accuracy.[21]

System	SST-2	CoLA	STS-B	MRPC	RTE	Average
	67k	8.5k	5.7k	3.5k	2.5k	-
Pre-OpenAL SOTA	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+EMMo+Attn	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	91.3	45.4	80.0	82.3	56.0	75.1
BERT-base	93.5	52.1	85.8	88.9	66.4	79.6
BERT-large	94.9	60.5	86.5	89.3	70.1	82.1

Table 2.1: GLUE Test results

2.2.4.4 Conclusion

Recent empirical improvements due to transfer learning with language models have demonstrated that rich, unsupervised pre-training is an integral part of many language understanding systems. In particular, these results enable even low-resource tasks to benefit from deep unidirectional architectures. Our major contribution is further generalizing these findings to deep bidirectional architectures, allowing the same pre-trained model to successfully tackle a broad set of NLP tasks. In a little over a year, BERT has become a ubiquitous baseline in NLP engineering experiments and inspired numerous studies analyzing the model and proposing various improvements. The stream of papers seems to be accelerating rather than slowing down, and we hope that this survey will help the community to focus on the biggest unresolved questions. [19]

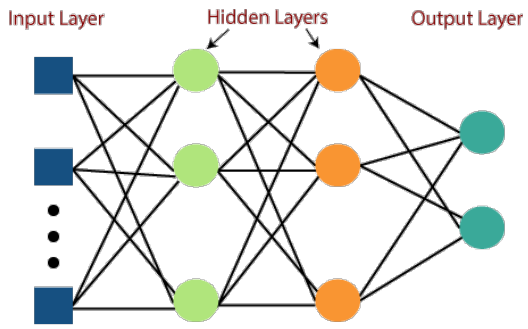


Figure 2.12: Multi-Layer Perceptron

2.2.5 Multi-Layer Perceptron

MLP is the basic imitation of a neural network, each artificial neuron in the network is called perceptron; there are layers of these perceptrons, these percpetrons perform mathematical computations layer by layer by activation functions and process the data layer by layer by training the weights of these layers; the basic intuition behind MLP was to solve complex, intelligent tasks by this artifical neural structure.The ability of tweaking and changing our data to be suitable for different type of neural structures allows us to solve multiple complex tasks from multiple domains. The ability to learn any mapping mathematically makes neural networks an application which is applicable in almost every circumstance. [11] Artifical neurons are present at every layer, they'll recieve data as input from previous layers and apply a mathematical function to it and pass the resultant value to the next layer, hence training the weights at each layer.

2.2.6 AutoEncoder

An autoencoder is a specific type of a neural network which is mainly designed to encode the input into a compressed and meaningful representation and then decode it back such that the reconstructed input is similar as possible to the original one.

An autoencoder is a special sort of neural network containing two parts encoder and a decoder; but these encoder and decoder models are based on the multi layered perceptron rather than the transformer model. The encoder neural network compress the data to a bottleneck, since it compresses the data to a bottle neck and then the decoder expands the

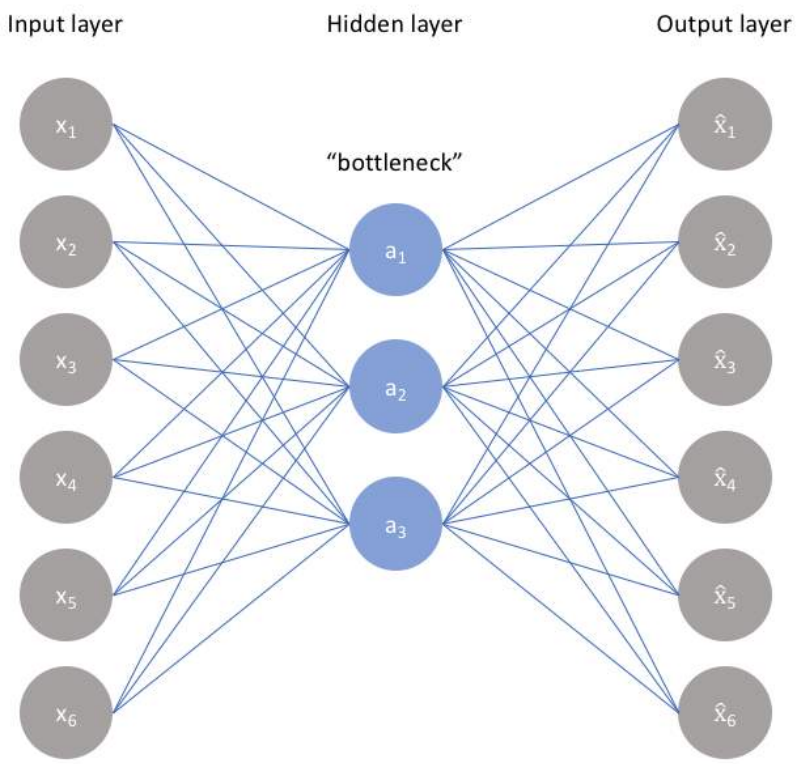


Figure 2.13: AutoEncoder

data from that bottleneck, it learns the representation and can be used for classification by taking a full trained encoder and adding classification layers on top it and retraining it. It is an unsupervised learning method which can be used for regeneration.

2.2.7 Variational Autoencoder

Variational Autoencoder is a special type of neural network which is basically a an autoencoder. What differs it greatly from a normal autoencoder is the last layer of it's encoder network, is compressed into two forms, mean and log sigma variance (which are layers themselves).The new image is generated from those two inputs. To use variational Autoencoder for classification, we'll take a full trained variational encoder and add multiple classification layers on top and retrain them to achieve a variational autoencoder.

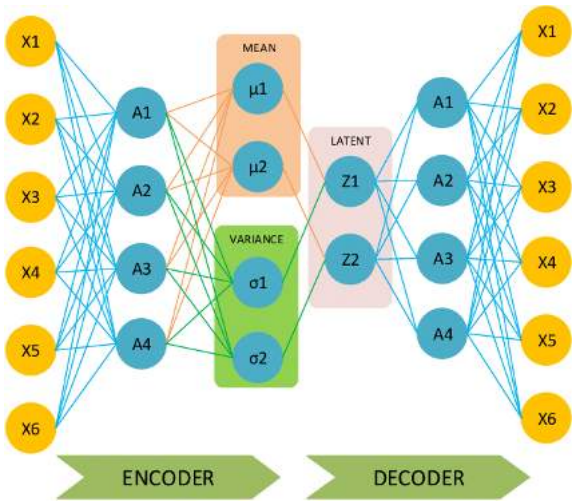


Figure 2.14: Variational AutoEncoder

2.3 Related Works and Background

Since the advent of Computers, Malwares or Malicious Applications are notorious for infecting personal computers and large scale systems; As technology is progressing and evolving rapidly at such a rate, the modern day Malicious applications are no far less behind. In order to keep up with the modern day attackers, modern day solutions are required; and the use of deep neural models has yielded optimum results in order to tackle a variety of modern day cyber security tasks.

2.3.1 Malware Analysis

Traditional anomaly detection schemes used old-school methods for malware detection such as rule based methods. The rule based methods would detect malware, as the name suggests by analysing the malware by some rules, modern day attackers have evolved their strategies to manipulate the modern day user of the internet. Hence to compete with one's with malicious intent, two types of malware analysis were used; Static Malware Analysis and Dynamic malware analysis.

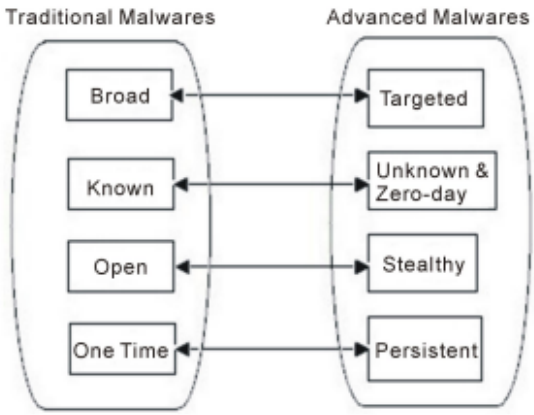


Figure 2.15: Traditional vs. advanced malwares.

2.3.2 Static Malware Analysis Vs Dynamic Malware Analysis

[14] In Static Malware Analysis, the malicious code will be analysed without being executed on the system, there are several methods of detection in this scheme can be based on string signature ,byte sequence n-grams, opcode (operational code) frequency distribution, etc. Modern day malicious applications tend to have obfuscation techniques applied to them. Obfuscation; means increasing the enigmity of your software code; so that your opponent isn't able to reverse engineer your code. This also leads certain malicious applications left undetected by anti-virus systems, thus altogether leaving static malware analysis to be very expensive and unreliable.

In order to analyse malicious code interacting with a system, we must construct a sandbox testing system; sandbox encompasses the malicious code to construct a safe environment for the malicious application to execute. One can analyse the activity sequences of the application to determine whether it is malicious or not; the activity sequences can be malicious sites contacted, detection can also be signature based.

2.3.3 Deep Neural Solutions

[20] achieved network activity sequences by the use of Wild-fire service by Palo Alto Networks, The Network Activity they recieved were in text format eg 'apk has hidden a file' etc. They conducted their experiments by constructing highly imbalanced datasets

with a sample of 0.5 percentage of Malware and achieved optimum results with the state-of-the art Neural Model Bert. [20] They also performed further analysis on the apk file data using Autoencoders, LSTMS but BERT stood out for their data. [13]

The works of [6] focuses on network level packet filtering Internet-of-Things malware detection based on the convolutional neural network; their method of collection was acquiring data from a local honeypot; achieving a overall F1 score 0.977 on a minute dataset.

[21] Modern day malware is beginning to be programmed to avoid network-traffic analysis; hence using the encrypted HTTPS protocol, [21] devised a protocol to classify malicious network flows using a long short-term memory (LSTM) networks

[15] experiment the task of malicious network activity detection with several variants of the state-of-the-art neural language BERT [9]. AlBert [17] or A-lite-Bert, chinese and english pre-trained Bert-base, and a Bert model pretrained on network traffic data were put up for the task of classification; Pre-trained Network Traffic Bert achieved the most optimum results.

[22] provide a solution to combat redundant alarm information. they provide us with the design of two-level-anomaly detection system. Apriori algorithm used to mine rules between various discretized features and normal labels, which can be used to filter the classified traffic and reduce the false positive rate. training was carried out on [22] NSL-KDD dataset.

The works of [24] Yirui Wu , Dabao Wei, and Jun Fen has discussed intelligent intrusion detection systems based on few generative models such as generative adversarial networks or GANS, AutoEncoders, They have also performed experiments on Deep Neural Network Models such as convolutional neural networks (CNNs) and Recurrent Neural networks (RNN), in a nutshell; discussing ways to achieve optimum results by maximizing the utility of Deep Learning Structures.

[12] discuss detection of Distributed Denial Of Service or DDoS using AutoEncoders and Deep Neural Networks, discussed the use of different activation functions at different layers to achieve optimum results.

[7] Amirah Alshammari and Abdulaziz Aldribi work primarily focuses on extracting fea-

tures used to train Machine Learning Models regarding a variety of attacks to determine whether it is anomaly or regular traffic. [10] The ISOT-CID dataset contains a type of flow which was first extracted from the network flow and the other which was computed in specific interval time, also introduced a new feature column to the dataset to increase chances of detection of malicious activity; that certain feature is also dependant on rambling packet payload length.

Chapter 3

Unified Modelling Language

This chapter provides diagrams designed for this project as a part of documentation.

3.1 Use Case Scenario

- System
  - System Collect Data
  - Preprocess the Data
  - Testing Preprocessed-Data (on trained DL model)
  - Notify User/Admin (on presence of malicious activity)
- User
  - Quarantine/Ban Malicious Nodes.

3.2 Use Case Diagram

This is Use Case Diagram of Malicious Network Activity detection using a Deep Neural Network, BERT.

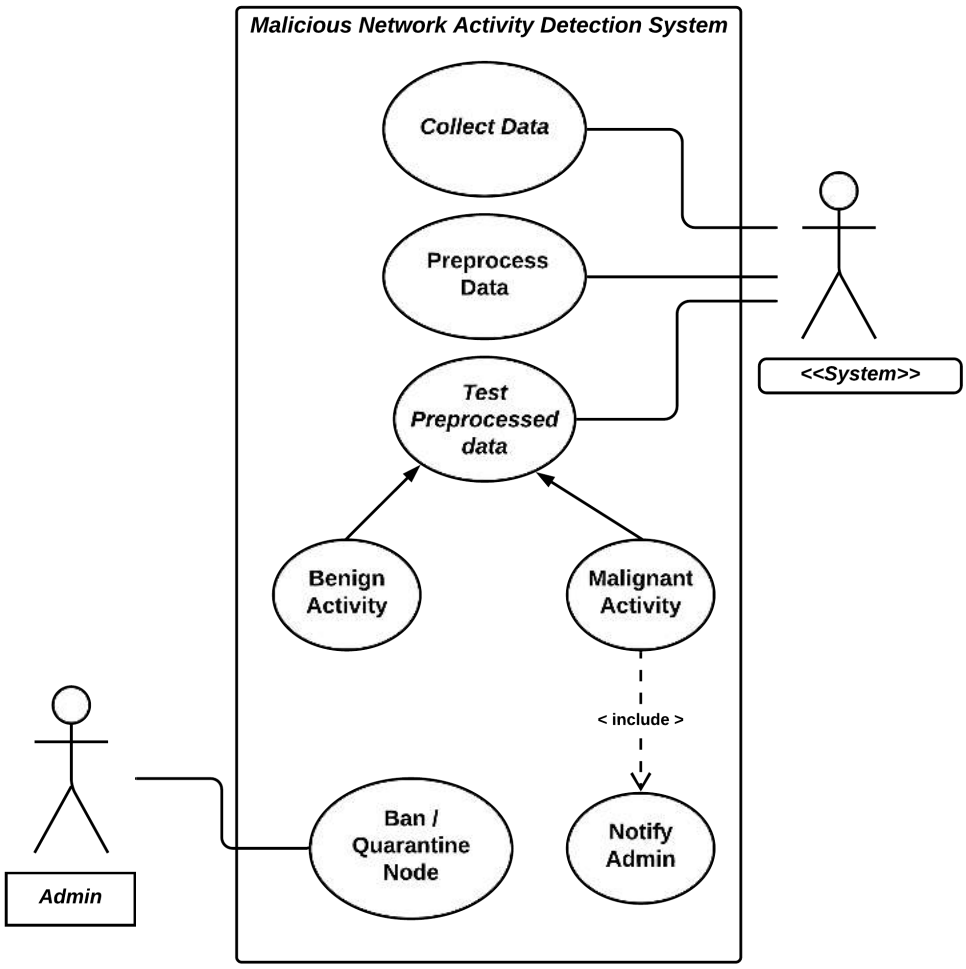


Figure 3.1: Use Case Diagram of Malicious Network Activity Detection

Figure 3.1 illustrates Use Case Diagram of Malicious Network Activity Detection, primary actor is Admin and secondary actor is «system». If we see the whole system we can see that we have use cases:

- Data Collection
- Data Preprocessing
- Testing Preprocessed-Data (on trained DL model)
- Notify User/Admin (on presence of malicious activity)

3.3 Activity Diagram

This is Activity Diagram of Malicious Network Activity detection using a Deep Neural Network, BERT

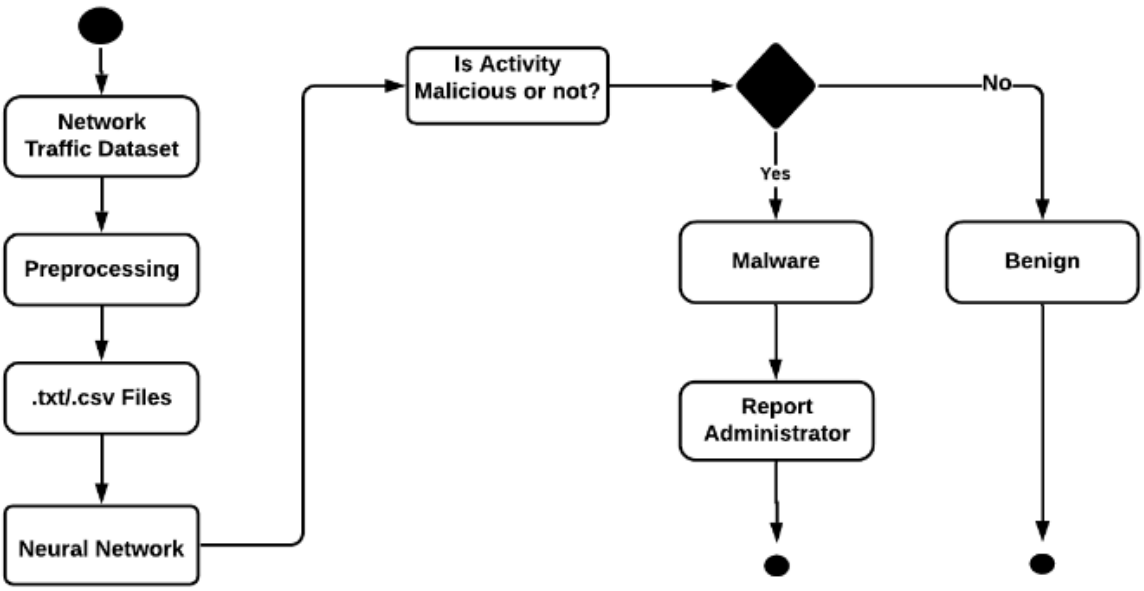


Figure 3.2: Activity Diagram of Malicious Network Activity Detection

As we can see Figure 3.2 above illustrates Activity Diagram of Malicious Activity Detection using Deep Neural Model. First of all we will capture Network traffic as a dataset,

after that we will preprocess the data and clean it such that we can fed in to Deep Neural Model and on detection of any malignant activity it should Report to administrator else flow end there. [21]

3.4 Class Diagram

This diagram represents class diagram of Malicious Network Activity Detection using Deep Learning.

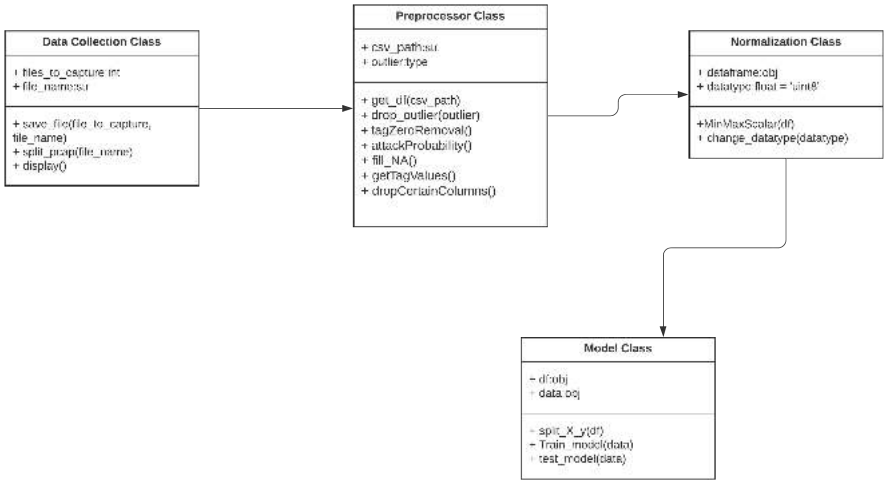


Figure 3.3: Class Diagram of Malicious Network Activity Detection

As Figure 3.3 shows we had four base class that are being used to manage the workflow, first of all, Data Collection class will handle real-time stream and write it to a file. Split pcap will split the bigger pcap file into smaller pcap files. Normalize class will normalized the data by checking datatypes and change all of them to unsigned int 8 and normalize the data such that it is in range of 0 to 255. After this, data is converted to HDF5 format and pass to Model class, this class have saved model weights to predict the data/ activity as malicious or benign.

Chapter 4

Methodology

This chapter demonstrates Methodology adopted in this project, the methodology, the experiments carried out to analyse and understand malicious network traffic data and our proposed system for Malicious Network Activity Detection using Deep Learning.

4.1 Model Preparation

4.1.1 Data Collection

We are using ICSX 2012 [2], ICSX 2017 [4] and ISCX 2018 [3] datasets for the purpose of training our model.We have extracted our data from multiple forms, such as writing a parser to parse data from XML files and extracting data from pcap files.

4.1.1.1 Data Parsing

As we were extracting data from multiple sources, we had to deal with each sort of data differently, 2012 data was easily available in the format of .xml files; hence to extract data from copious amounts of XML files; we constructed a Parser class using etree module to collect and parse data from that source. As shown in Source Code A.1

4.1.1.2 Excerpt of Parser Class

```
from lxml import etree

# xml parser object
doc = etree.iterparse('2012\TestbedSunJun13Flows.xml',  events=('start',), tag='TestbedSunJun13Flows' )
print(doc)

# Appending header row into csv file
filename =  etree.iterparse('2012\TestbedSunJun13Flows.xml', events=('start',), tag='TestbedSunJun13Flows' )


class Parser:

  def __init__(self):
    self.doc = etree.iterparse(filename,  events=('start',), tag='TestbedSunJun13Flows' )
    self.csv = []
    with open('flowSunJun115.csv', 'a') as f: #Write CSV File
        header =
        'appName,source,protocolName,sourcePort,destination,destinationPort,direction,totalSourceBytes,totalDestinationBytes,totalDestinationPackets,totalSourcePackets,sourcePayloadAsBase64,sourcePayloadAsUTF,destinationPayloadAsBase64,destinationPayloadAsUTF,sourceTCPFlagsDescription, destinationTCPFlagsDescription,startDateTime,stopDateTime,Tag'

        f.writelines(header+"\n")


  def write_as_csv(self,row):
      """
          write dictionary into csv
      """
```

```
        r = ' , '.join(list(row.values()))

        with open('flowSunJun115.csv', 'a') as f:
            f.writelines(r+"\n")
        print(r)
```

Source Code 4.1: Excerpt of Parser Class

4.1.2 Data Preprocessing

As the data we had collected is network activity data, that data proved to have numerous inconsistencies, such as missing rows, null values and infinity values; hence a Preprocessor() class was constructed to attain perfect data to perform experiments on. As shown in Source Code A.2

4.1.2.1 Excerpt of Preprocessor Class

```
import threading

'''
    Preprocessor class:
    this is the class which will be used to manipulate our data.
    -> self.df; is the member; in which the dataframe will be
   loaded.
    -> Preprocessor: is used to clean & pre-process our data.

'''

class Preprocessor:

    def __init__(self, csv_path=filename):
```

```
        #Reading the csv file pn creation & loading it in dataframe.
        self.df = pd.read_csv(csv_path)

    def get_df(self):
        #Returns DataFrame
        return self.df

```

Source Code 4.2: Excerpt of Preprocessor Class

4.1.3 Feature Engineering

After attaining our dataset, we passed our data from principal component analysis to perform dimensionality reduction and then we mapped each attack label to benign with the use of Kmeans clustering algorithm. We learned that the data wasn't easily separable for Kmeans. We used RandomForestRegressor and Extremely Randomized trees or extra trees to perform feature selection and were successful in attaining the most important features for classification; and hence reduced our features from 80 to 55.

4.1.4 Experiment's Setup

Experiment setup consist of many free and open-source software library and tools. Remember we want to classify whether captured real-time network activity is malignant or benign. So, for that initially we need a tool to capture real-time network traffic for that we used Wireshark command-line utility called Dumpcap, once network stream is captured we need a tool to convert pcap file to csv file for that we used splitcap (to split massive gigabyte pcap to smaller once) and PRC Flowmeter (pcap to csv file conversion). Now we use several implementation of machine learning/ deep learning model in the lights of Sci-kit learn, Tensorflow 2, Keras and Pytorch, used all of them for training and testing of machine learning/ deep learning models. We used JupyterLab and google Colab and kaggle resources (with GPU for computational resorces).

4.1.5 Experiment's Performed

We performed several experiments to check the performance of each classifier on different type of data, and after conducting our experiments we have found, the size of each target label distribution (which is quantity of different attacks in our case), the datatype of data (unsignedint or float.64) all make a difference in the accuracy of our model. All of these changings in data reflect a changing in the performance of the model, we have constructed two datasets to train our model by.

4.1.6 Dataset Analysis

We performed data analysis on the total dataset we had to get a feel of the data and to understand the intricacies which differentiate a malicious network stream from a benign one. We saw the relations between certain flag types to the label of the class, after performing this analysis, we were able to see the certain features which were differentiating a benign stream from a malicious one.

4.1.6.1 Fin Flag

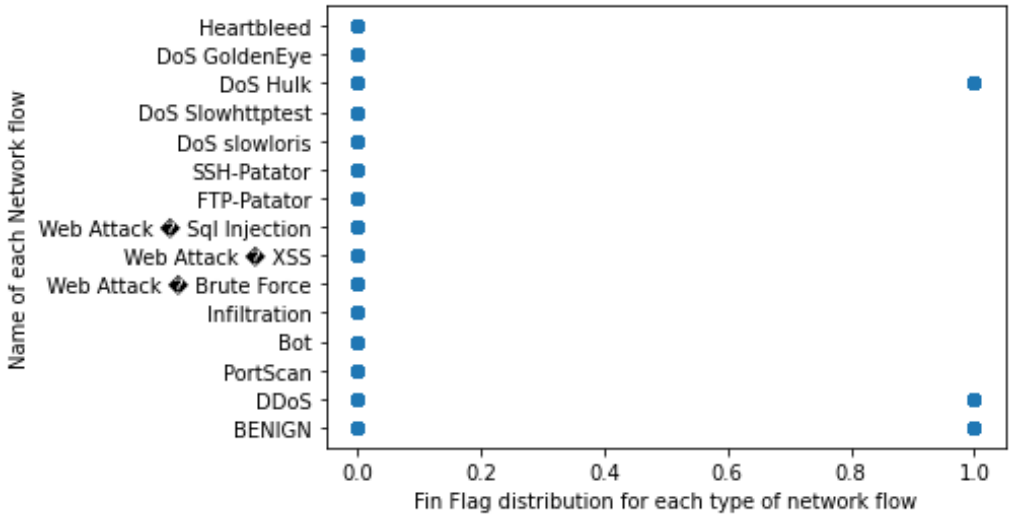


Figure 4.1: Acknowledgement flag values to each type of stream

In this plot, we can see the distribution of the value of Fin Flag for each type of stream. Fin Flag is set when a communication between two nodes on the network is being terminated. We can see most attacks don't have value for the fin flag set. Distributed Denial of service and Dos Hulk are the only two type of malicious streams which have the value for the fin flag to be set. This is whst Benign, Dos Hulk snd DDos streams have in common.

4.1.6.2 Synchronize Flag

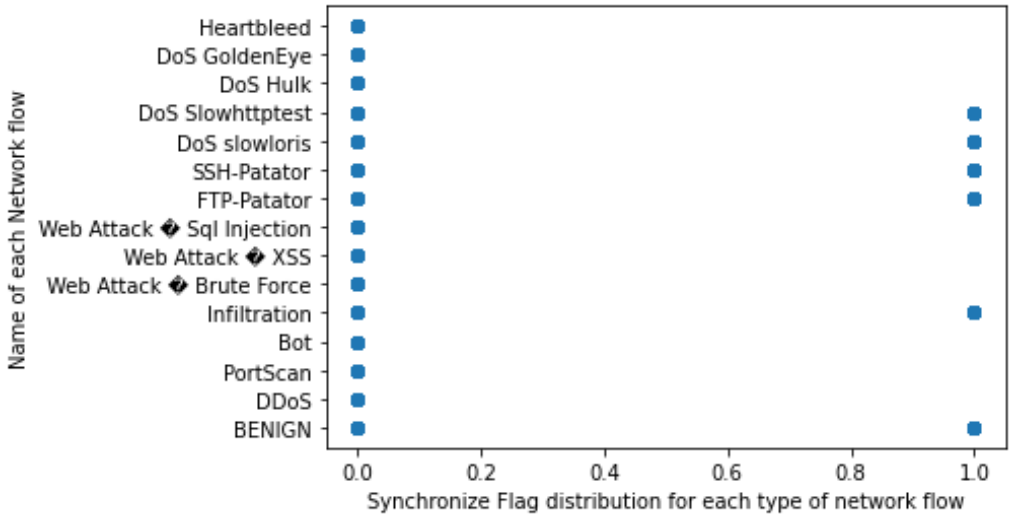


Figure 4.2: Synchronize flag values to each type of stream

Synchronize or SYN flag is used in the TCP protocol to mark the formation of a connection between two hosts. The synchronization flag is set only in the first packet of both the source and destination ports. According to this plot, we can see the difference in setting of the flag for each type of stream. Benign streams can have it's flag set to both zero and one, where as in distributed denial of service, portscan, botnets, web attacks, heartbleed attacks will have the syn flag to be set to zero only. Attacks such as brute force, Patator attacks have the values of synchronization flag set as zero and one like benign streams.

4.1.6.3 Acknowledgment Flag

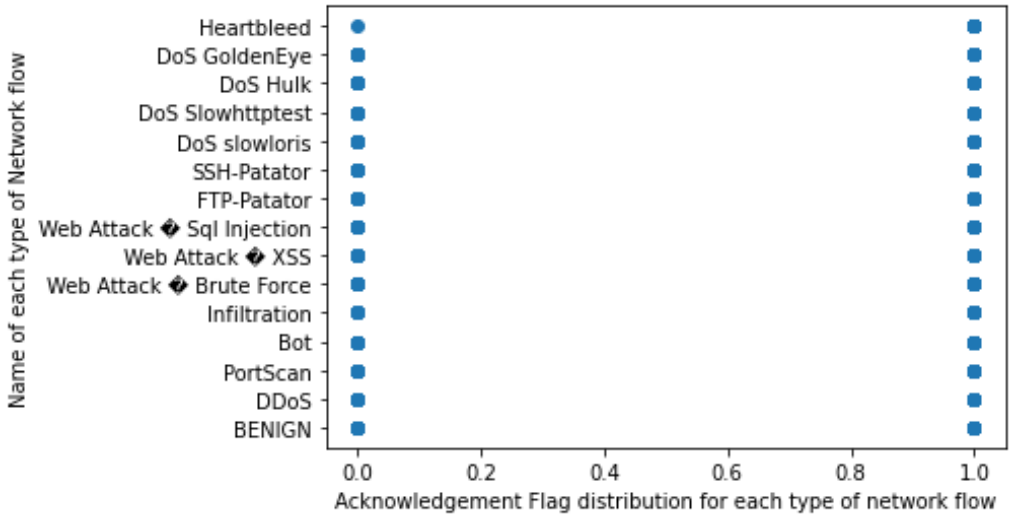


Figure 4.3: Density Plot of Rest Flag to each type of stream

Ack flag or Acknowledgment flag as the name suggests gives sets to give confirmation of formation of connection, it is set when communication gets established. As we can see in this plot, the acknowledgement flag is set in each sort of stream, every sort of stream's network packet may contain acknowledgement flag value to be set or reset. As, we can see acknowledgement flag isn't the measure to differentiate between different type of streams, as the behaviour of acknowledgement flag is identical for every stream.

4.1.6.4 Push Flag

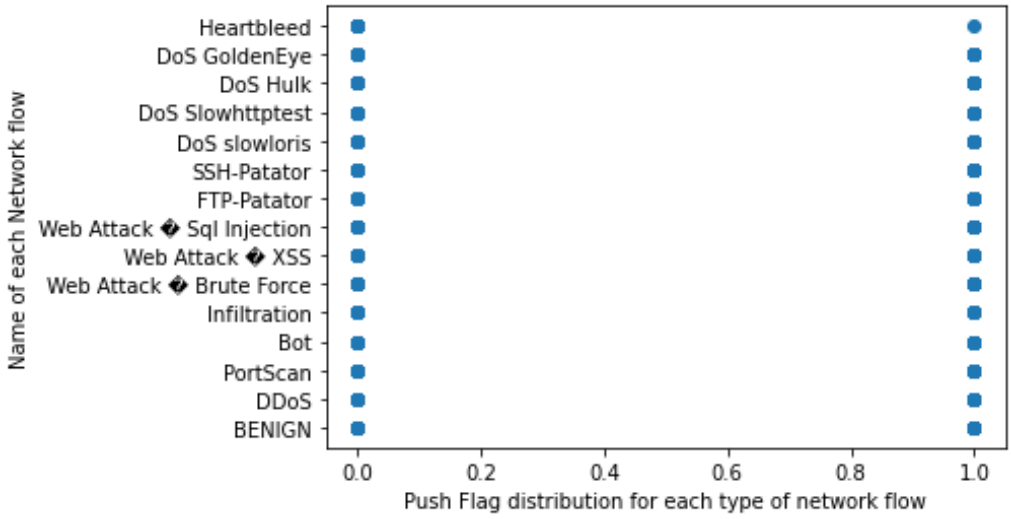


Figure 4.4: Density Plot of Push Flag to each type of Network Flow

Push flag as the name suggests gives sets to give confirmation of formation of connection, it is set when communication gets established. As we can see in this plot, the push flag is set in each sort of stream, every sort of stream's network packet may contain acknowledgement flag value to be set or reset. As, we can see push flag isn't the measure to differentiate between different type of streams, as the behaviour of push flag is identical for every stream, as push flag is set when the sending application informs the tcp protocol that data is to be set, and informs the application the sending data is to be passed up to the recieving application.

4.1.6.5 CWE Flag

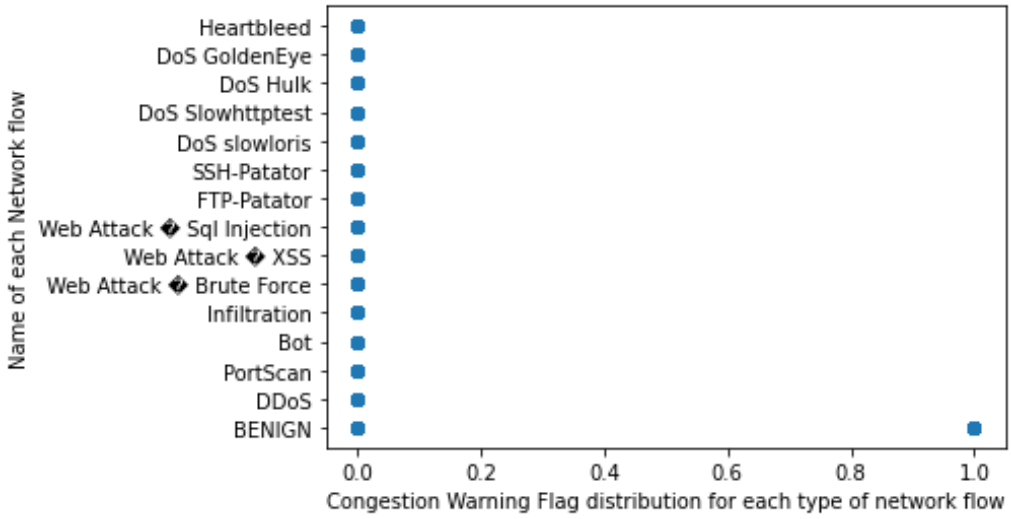


Figure 4.5: Density Plot of Congestion Warning Flag to each type of Network Flow

Congestion warning flag is as the protocol senses congestion ahead, congestion occurs when a certain network is inadequate in managing traffic in that certain network, congestion can lead to packet loss, as we can see from this distribution that only a benign stream will set the congestion warning to one in chances of congestion. Malicious streams such as distributed denial of service, portscan, etc. don't warn the network about the congestion.

4.1.6.6 Urgent Flag

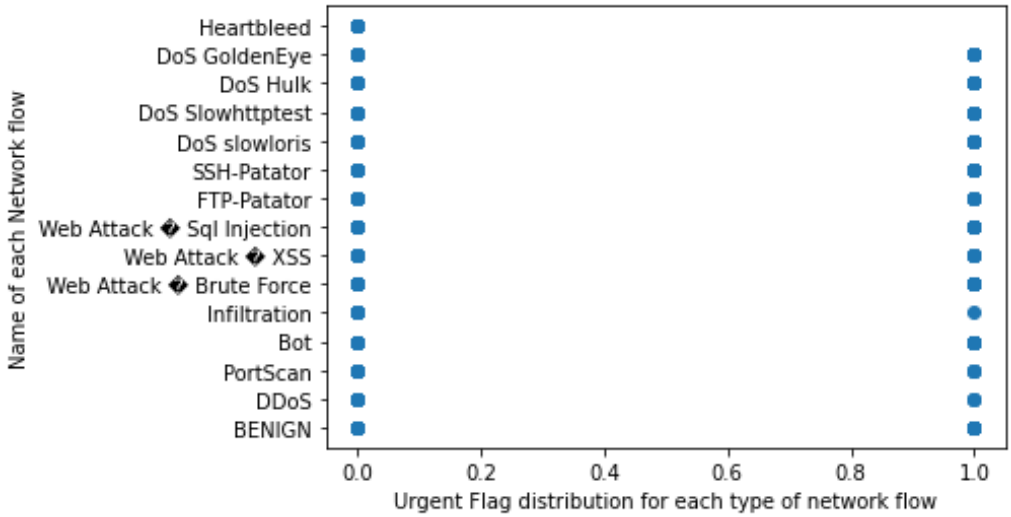


Figure 4.6: Density Plot of Congestion Warning Flag to each type of stream

The urg flag or urgent flag is set when the recieving node has to prioritize the recieving data. As, we can see urgent flag isn't the best measure to differentiate between different type of streams, as the behaviour of urgent flag is identical for every stream except heart bleed attacks, as urgent flag is set when the recieving application has to prioritize the data it is recieving.

4.1.6.7 Number of packets transferred per second

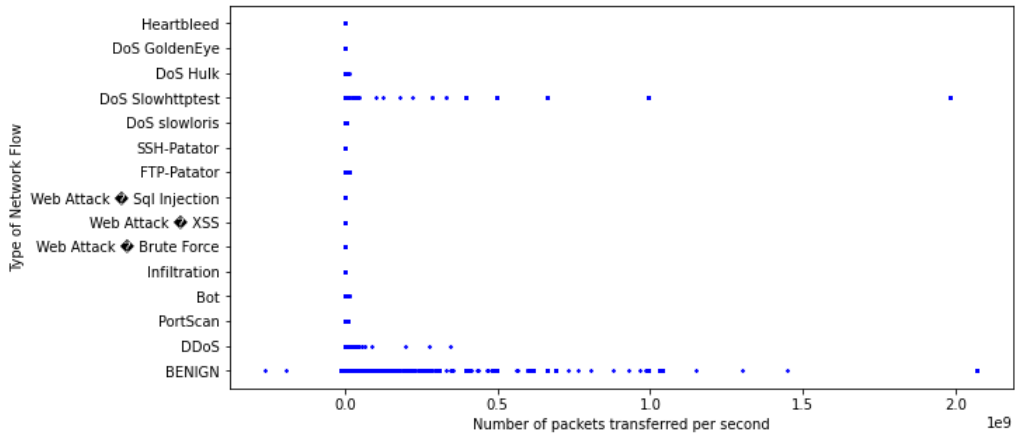


Figure 4.7: Distribution of flow duration to each stream

The plot above shows us the difference in number of packets transferred per second in benign and malicious streams. As we can see from this plot, Benign nodes in a network's number of packets transferred in a second is much hugher than distributed denial of service; As in DDos multiple machines are transferring a large number of packets, not only one machine. Denial of service slowhttptest also has a large number of packets transferred per second.

4.1.6.8 Destination Ports Attacked

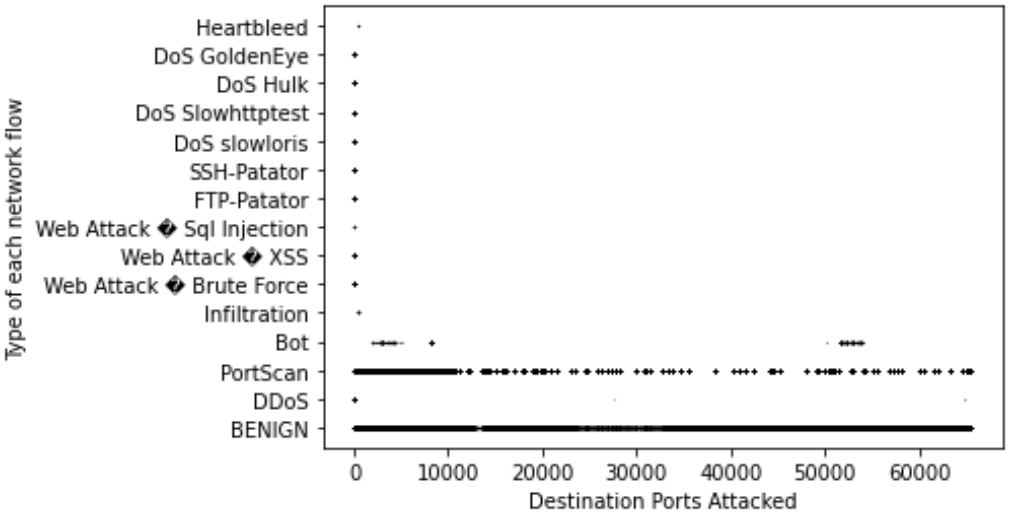


Figure 4.8: Distribution of flow duration to each type of stream

The plot above shows us a clear difference in the distribution of destination ports attacked in a benign stream and a malicious stream. The benign stream sends data on every available port, there a total of 65,535 ports, where as rest of the attacks operate on certain ports, Botnet is shown to occur on a little variety of ports. Portscan Attack which is known to scan vulnerable ports and then perform mitigation's via that port is scanning a large distribution of ports.

4.1.6.9 Flow duration of each sort of flow

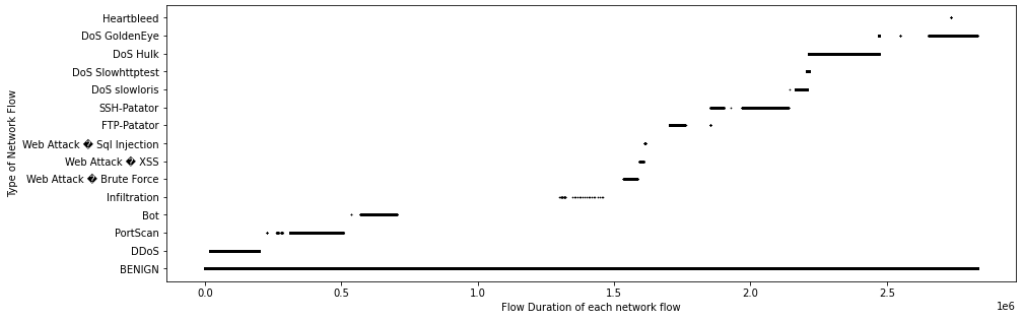


Figure 4.9: Distribution of flow duration to stream

The plot above shows us how long each type of stream mantain communication, Benign streams are shown to operate at every duration where as distributed denial f service is shown to communicate at the initial durations of the connection, Portscan and botnet are shown to hold a longer flow duration than every distributed denial of service attack. Cross site scripting attacks hold a minute flow duration, Distributed Denial of service GoldenEye is shown to hold one of the longest flow durations for malicious streams.

4.1.6.10 Dataset Construction

we constructed two definitive datasets for testing and training our models, Dataset A contains data in the float.64 datatype and the data isn't normalized or scaled; we have found this dataset enables our models to perform better, but since the dataframe size becomes too large, it increases training time but vastly improves our performance e.g, Increased MLP F1 score from 0.81 to 0.998. Dataset B contains data with datatype unsigned8, and all feature values scaled to 255, this decreases our accuracy but also decreases training time.

4.1.6.11 Dealing with Highly Imbalanced Data

To mimic the nature of real life malicious network traffic, we constructed our dataset with 25 percent malicious data, since we increased the quantity of both malicious to benign traffic but mantaining the 25:75 ratio allowed neural nets such as MLP (Multi-Layer Perceptron) and AutoEncoders to still mantain optimum results. As we can see this distribution, some attacks are very low in number, to enable our deep neural network to achieve the best possible results, Dataset A and Dataset B were constructed; both mantained the 25:75 ratio mentioned above.

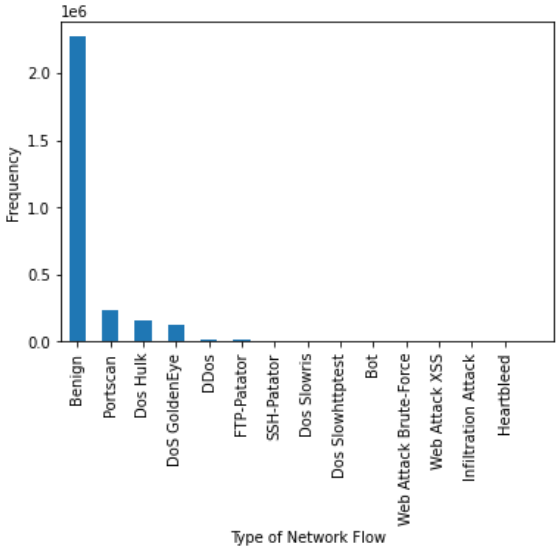


Figure 4.10: Acknowledgement flag values to each type of Network Flow

4.1.6.12 Models

We put the science of Machine Learning and Deep Learning to the test of analysing Network Traffic Data, and we can proudly claim to achieve optimum results with a variety of Deep Neural Architectures and Machine Learning Models. These are the Machine Learning and Deep Learning Models that we have used in our experiments.

1. Decision Tree Classifier
2. RandomForest Classifier
3. Extremely Randomized Trees
4. Naive Bayes
5. K-Means
6. Logistic Regression
7. Multi-Layered Perceptron (MLP)
8. Bidirectional Encoders Representation through Transformers (BERT)
9. AutoEncoders (AE)
10. Variational AutoEncoders (VAE)

4.2 Workflow Sequence

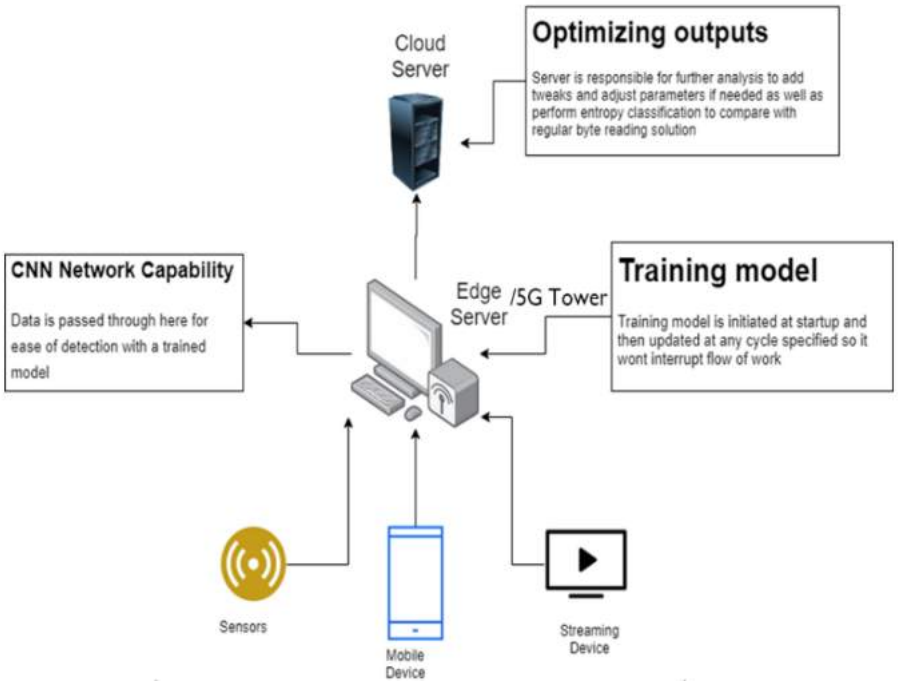


Figure 4.11: Work Flow of Malicious Network Activity Detection

4.2.1 Automated Data Collection

We previously said that we would collect/capture a real-time stream, which would then be executed as an automated script and sent through an trained model to determine if the stream (packets) were malicious or benign. We wanted to collect a stream that was coming in real-time for this. We needed to design such a mechanism that we can capture network traffic from live network and dump it into a file so that data can be used to classify our labels.

4.2.1.1 Dumpcap

As a result, we employed Dumpcap, a wireshark command-line utility. It's a tool for dumping network traffic. It allows you to capture live network packets and dump them to a file. Dumpcap captures packets in pcapng format, however if the -P flag/ option is given,

it outputs in pcap format. Given below Figure 4.1 that shows how we can use dumpcap to capture network stream through a live network and dump into a pcap file, where '-i' option let you select interface on which we want to capture packet and '-b' option specify command ending criteria like time till capture packets or filesize.

```
dumpcap -i 1 -w file.pcap -b filesize:50
➜ ~ dumpcap -i 1 -w file.pcap -b filesize:50
Capturing on 'wlp1s0'
File: file_00001_20211213225738.pcap
Packets: 171 File: file_00002_20211213225810.pcap
Packets: 321 File: file_00003_20211213225811.pcap
Packets: 475 File: file_00004_20211213225826.pcap
Packets: 569 File: file_00005_20211213225826.pcap
Packets: 611 File: file_00006_20211213225826.pcap
Packets: 654 File: file_00007_20211213225826.pcap
Packets: 696 File: file_00008_20211213225827.pcap
Packets: 739 File: file_00009_20211213225827.pcap
Packets: 783 File: file_00010_20211213225827.pcap
Packets: 827 File: file_00011_20211213225827.pcap
Packets: 874 File: file_00012_20211213225827.pcap
Packets: 917 File: file_00013_20211213225827.pcap
Packets: 960 File: file_00014_20211213225827.pcap
Packets: 1003 File: file_00015_20211213225827.pcap
Packets: 1045 File: file_00016_20211213225827.pcap
Packets: 1088 File: file_00017_20211213225827.pcap
Packets: 1130 File: file_00018_20211213225827.pcap
Packets: 1173 File: file_00019_20211213225827.pcap
Packets: 1219 File: file_00020_20211213225827.pcap
Packets: 1261 File: file_00021_20211213225827.pcap
Packets: 1308 File: file_00022_20211213225827.pcap
```

Figure 4.12: Dumpcap illustration

If no option is specified, dumpcap will collect data from the first available network interface and dump the stream to a file with packet time stamps using the libpcap, Npcap, or WinPcap libraries. If the -w parameter is given, the pcap file will be saved in the allotted place; otherwise, it will be created with the random name. The pcap library is used for packet capture. The capture filter syntax follows the pcap library's standards.[23]

4.2.1.2 What is PCAP File Format?

Packet Capture file or pcap file are used for monitoring network traffic and analysing network traffic data so that it's easier for humans to understand it. We can use pcap to monitor networks for a variety of reasons, we can detect malware, resolve DNS, detect rogue DHCP servers by the use of pcap captures. Monitoring pcap files is an effective way to identify compromised networks and other activities which the network manager, system administrator should know. To capture pcap files, we need to use a packet sniffer, to utilisize a packet sniffer, we must determine which interface we're going to capture data from. We can select which sort of data we want to capture, several tools such as wireshark, tshark, tcpdump, dumpcap can be used to capture real-time-stream.

No.	Time	Source	Destination	Protocol	Length	Info
1503944	7319.870392	82.98.86.183	192.168.2.112	TLSv1	1085	Application Data
1503945	7319.875275	82.98.86.183	192.168.2.112	TCP	1514	443 → 3066 [ACK] Seq=16864 Ack=680 Win=7504 Len=1460 [TCP segment of a reassembled PDU]
1503946	7319.876502	192.168.2.112	82.98.86.183	TCP	60	3066 → 443 [ACK] Seq=680 Ack=18324 Win=64240 Len=0
1503947	7319.878773	82.98.86.183	192.168.2.112	TCP	1514	443 → 3066 [ACK] Seq=18324 Ack=680 Win=7504 Len=1460 [TCP segment of a reassembled PDU]
1503948	7319.886406	82.98.86.181	192.168.2.112	HTTP	546	HTTP/1.0 200 OK (GIF89a)
1503949	7319.887600	192.168.2.112	82.98.86.181	HTTP	171	GET /fileadmin/templates/sedo/gfx/logos/languages/flag_nl.gif HTTP/1.1
1503950	7319.922035	192.168.2.107	98.113.151.116	TCP	122	[TCP Retransmission] 3291 → 58008 [FIN, PSH, ACK] Seq=1 Ack=2 Win=17520 Len=68
1503951	7319.960918	87.202.84.33	192.168.2.107	UDP	62	21184 → 58040 Len=20
1503952	7319.998280	192.168.2.112	82.98.86.183	TCP	60	3066 → 443 [ACK] Seq=680 Ack=19784 Win=64240 Len=0
1503953	7320.007189	82.98.86.183	192.168.2.112	TLSv1	1514	Application Data [TCP segment of a reassembled PDU]
1503954	7320.008401	192.168.2.112	82.98.86.183	TCP	60	3066 → 443 [ACK] Seq=680 Ack=21244 Win=64240 Len=0
1503955	7320.011154	82.98.86.183	192.168.2.112	TCP	1514	443 → 3066 [ACK] Seq=21244 Ack=680 Win=7504 Len=1460 [TCP segment of a reassembled PDU]
1503956	7320.014935	82.98.86.183	192.168.2.112	TCP	1514	443 → 3066 [ACK] Seq=22704 Ack=680 Win=7504 Len=1460 [TCP segment of a reassembled PDU]
1503957	7320.014941	94.178.133.49	192.168.2.107	TCP	60	51000 → 3406 [ACK] Seq=664 Ack=664 Win=64873 Len=0
1503958	7320.016158	192.168.2.112	82.98.86.183	TCP	60	3066 → 443 [ACK] Seq=680 Ack=24164 Win=64240 Len=0
1503959	7320.018688	82.98.86.183	192.168.2.112	TLSv1	1514	Application Data [TCP segment of a reassembled PDU]
1503960	7320.019900	192.168.2.112	82.98.86.183	TCP	60	3066 → 443 [ACK] Seq=680 Ack=25624 Win=64240 Len=0
1503961	7320.022426	82.98.86.183	192.168.2.112	TCP	1514	443 → 3066 [ACK] Seq=25624 Ack=680 Win=7504 Len=1460 [TCP segment of a reassembled PDU]
1503962	7320.026423	82.98.86.183	192.168.2.112	TCP	1514	443 → 3066 [ACK] Seq=27084 Ack=680 Win=7504 Len=1460 [TCP segment of a reassembled PDU]
1503963	7320.027642	192.168.2.112	82.98.86.183	TCP	60	3066 → 443 [ACK] Seq=680 Ack=28544 Win=64240 Len=0
1503964	7320.028307	82.98.86.183	192.168.2.112	TLSv1	1107	Application Data, Application Data, Encrypted Alert
1503965	7320.029234	192.168.2.112	82.98.86.183	TCP	60	3066 → 443 [ACK] Seq=680 Ack=29598 Win=63187 Len=0
1503966	7320.029681	192.168.2.112	82.98.86.183	TCP	60	3066 → 443 [FIN, ACK] Seq=680 Ack=29598 Win=63187 Len=0
1503967	7320.033279	82.98.86.181	192.168.2.112	HTTP	498	HTTP/1.0 200 OK (GIF89a)
1503968	7320.107618	192.168.2.112	82.98.86.181	TCP	60	3067 → 80 [ACK] Seq=590 Ack=2811 Win=62898 Len=0
1503969	7320.125153	192.168.2.107	93.85.169.114	TCP	122	[TCP Retransmission] 3293 → 51413 [FIN, PSH, ACK] Seq=1 Ack=2 Win=16800 Len=68
1503970	7320.169282	82.98.86.183	192.168.2.112	TCP	60	443 → 3066 [ACK] Seq=29598 Ack=681 Win=7504 Len=0
1503971	7320.171974	192.168.2.107	94.232.14.67	UDP	62	58040 → 64525 Len=20
1503972	7320.273249	192.168.1.103	209.202.254.60	TCP	60	[TCP Retransmission] 3903 → 80 [FIN, ACK] Seq=922 Ack=13040 Win=17056 Len=0
1503973	7320.286994	192.168.2.112	91.195.240.124	HTTP	175	GET /fileadmin/templates/sedo/gfx/buttons/default/close_popup.gif HTTP/1.1
1503974	7320.296269	192.168.2.106	91.190.170.71	TCP	60	[TCP Retransmission] 3712 → 80 [FIN, ACK] Seq=1388 Ack=134732 Win=17412 Len=0

Figure 4.13: raw pcap file containing network packets from which we are extracting our dataset.

4.2.1.3 Splitcap

As we start capturing the real time network stream, as it is a network stream, data can get messy and we can extract a lot of unwanted stuff; so to recieve structured data; we can use. [1]SplitCap is a tool which allows us to capture a pcap file on the basis of 5-tuple.

The criteria for splitting/ grouping are as follows:

- **BSSID:** Packets created on the basis of WLAN BSSID
- **Flow:** 5-tuple (transport protocol, IP addresses and port numbers) are grouped together by Unidirectional traffic.
- **Host:** Traffic grouped on the basis of IP address (source and destination) for each file. The majority of packets will be split into two files.
- **Host Pair:** Traffic is divided into groups based on the IP pairs that are interacting.
- **MAC address:** Per MAC address, traffic is organised into one file. The majority of packets will be split into two files.
- **Session:** Each session's packets are pooled together (bi-directional flow).
- **Time:** The division is based on the passage of time.
- **Packets Count:** Split based on the number of packets.

```
Usage: SplitCap [OPTIONS]...

OPTIONS:
-r <input_file> : Set the pcap file to read from. Use "-r -" to read from stdin
-o <output_directory> : Manually specify output directory
-d : Delete previous output data
-p <nr_parallel_sessions> : Set the number of parallel sessions to keep in memory (d
efault = 10000). More sessions might be needed to split pcap files from busy links s
uch as an Internet backbone link, this will however require more memory
-b <file_buffer_bytes> : Set the number of bytes to buffer for each session/output f
ile (default = 10000). Larger buffers will speed up the process due to fewer disk wr
ite operations, but will occupy more memory.
-s <GROUP> : Split traffic and group packets to pcap files based on <GROUP>. Possibl
e values for <GROUP> are:
  bssid : Traffic grouped based on WLAN BSSID
  flow : Each flow, i.e. unidirectional traffic for a 5-tuple, is grouped
  host : Traffic grouped to one file per host. Most packets will end up in two file
s.
  hostpair : Traffic grouped based on host-pairs communicating
  mac : Traffic grouped to one file per MAC address. Most packets will end up in two
files.
  nosplit : Do not split traffic. Only create ONE output pcap.
  (default) session : Packets for each session (bi-directional flow) are grouped
  seconds <s> : Split on time, new file after <s> seconds.
  packets <c> : Split on packet count, new file after <c> packets.
-ip <IP address to filter on>
-port <port number to filter on>
-y <FILETYPE> : Output file type for extracted data. Possible values for <FILETYPE>
are:
  L7 : Only store application layer data
  (default) pcap : Store complete pcap frames

Example 1: SplitCap -r dumpfile.pcap
Example 2: SplitCap -r dumpfile.pcap -o session_directory
Example 3: SplitCap -r dumpfile.pcap -s hostpair
Example 4: SplitCap -r dumpfile.pcap -s flow -y L7
Example 5: SplitCap -r dumpfile.pcap -s seconds 3600
Example 6: SplitCap -r dumpfile.pcap -ip 1.2.3.4 -port 80 -port 443 -s nosplit
Example 7: SplitCap -r C:\pcaps\ -recursive -s host -port 53 -o DNS_dir
Example 8: tcpdump -n -s0 -U -i eth0 -w - | mono SplitCap.exe -r -
```

Figure 4.14: Splitcap illustration

Figure 4.2 show all the 'OPTIONS' we can use to split mega gigabyte pcap file into smaller pcap files by choosing desired criteria for splitting/ grouping, figure also listed some examples to illustrate the concept behind using splitcap.

4.2.2 Automated Conversion

4.2.2.1 PRC Flowmeter

We have divided the pcap file and can now convert it to csv format using PRC Flowmeter v0.2.0, a network traffic flow generator and analyzer. For data analysis and machine learning, Flowmeter is a Scapy-based application for extracting statistical characteristics from PCAPs.[5] This tool's features are inspired by CICFlowmeter java application. Up to three parameters can be used to build a Flowmeter object:

- **offline (str)** - pcap filename, if not given, streams from available ports
- **outfunc (function)** - As the metered flows are produced, a csv rendition will be supplied to this function. Defaults to print() if none is specified.
- **outfile (str)** - filename to same csv flow output, if not provided results are not stored.

With scapy Sessions, Flowmeter divides packet streams into discrete network communication 'flows,' which are defined as communication between two endpoints (ip:port) on a specified protocol during a specific period of time. It then begins analysing the flow data in order to extract characteristics that may be used for plotting, traffic pattern analysis, and machine learning.[5]

4.2.3 Automated Feature Scaling

When it comes to the implementation of various Machine Learning Algorithms, data preprocessing and normalizing become quite critical. Because data preparation can have a substantial impact on the learning model's output, it's critical that all features are on the same size. In models like k-NN, support vector machines, neural networks, and principal components, normalisation is crucial. Depending on the data, the type of feature preprocessing and normalisation requirements may vary.

Many Machine Learning methods rely heavily on data transformation. The appropriate form of features normalisation to utilise is determined on the data set, learning goal, and learning algorithm. When it comes to data normalisation and standardisation, there is no hard and fast rule. To get the best results, start by fitting the model to raw data, normalised and standardised data, and comparing the best results.

4.2.3.1 Automated Preprocessing

We have a network traffic stores in csv format, now next step is to preprocess and normalize the captured data. We have selected 55 features from dataset. We preprocessed and

normalization each feature. We normalization all feature and set them 0 to 255 range. We change the datatype of all features to 'uint8' so it that less time in loading and processing. We applied all these preprocessing and normalization techniques and convert dataset into HDF5 format to fed to machine learning or deep learning models.

4.2.3.2 Automated Normalization and Standardization

Normalization is a scaling method that shifts and rescales data to make them range between 0 and 1. it is also called Min-Max scaling.

Formula for Normalization:

$$X^{'} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Xmax and Xmin are the feature's maximum and minimum values, respectively. When the value of X is the minimum in the column, the numerator is 0, and so X' is 0. When the value of X, on the other hand, is the maximum value in the column, the numerator equals the denominator, and so the value of X' is 1. If the value of X is between zero and one, then the value of X' is between zero and one. On the training data, the method computes the minimum and maximum values for each feature, and then applies the min-max transformation to each feature.

Standardization is another scaling technique, in which the values are centered around the mean with an uniform standard deviation. As a result, the attribute's mean becomes zero, and the resulting distribution has a unit standard deviation.

Formula for Standardization:

$$X^{'} = \frac{X - \mu}{\sigma}$$

The mean of the feature values is denoted by 'u', while the standard deviation is denoted by 'Sigma'. It should be noted that the values in this situation are not limited to a certain range. StandardScaler ensures that the mean and variance of each feature in the dataset

are both 0 and 1, bringing all features to the same magnitude. The features' minimum and maximum values are not specified by this scaling.

4.2.4 Automated Model Prediction

Now we have preprocessed and normalized data in hdf5 format. We had train certain machine learning and deep learning models and saved their weights in hdf5 format in google drive, so we loaded model weights and predicted the accuracy score. Now our model is ready to predict, so we pass it predictive data (captured from live network, converted to csv format, and then finally in hdf5 format), and classify labels as malicious or benign.

4.2.5 Maintain Activity log file

Our Deep Neural Network had correctly classify malignant and benign network packets, so now to notify about information gained from this classification model to system admin we need to dump all the predicted labels to a log file. The system administrator doesn't need to interact with a front end, so an application continously updating the log file would be easier in managing; then the system administrator can mitigate the node according to his will.

Chapter 5

Results

This chapter illustrates the results we achieved for Malicious Network Activity Detection using Deep Learning and Machine Learning models. As we had previously mentioned, we had divided our dataset into two parts

5.1 Results

5.1.1 Evaluation Metrics

5.1.2 Precision:

$$Pr = \frac{TP}{TP + FP}$$

5.1.3 Recall:

$$Rc = \frac{TP}{TP + FN}$$

5.1.4 F1 Score:

$$F1 = \frac{2}{\frac{1}{Pr} + \frac{1}{Rc}}$$

5.1.5 Datasets

In order to perform our experiments in such a way which allow us to understand the intricacies which link cyber-security and deep learning, and also understand the neural structures on which we're performing the experiments on. These experiments also enabled us to understand how each specific model would react to on different kinds of data.

5.1.6 Dataset A

Dataset A contains data in the rawest, preprocessed form, hence preserving the true shape of data, (eg. as there are a total of 65,353 ports, the port feature will have 65,535 different values). Dataset A is the dataset which contains a total of 800,000 samples:

1. 650,000 Benign Streams
2. 50,000 DDos Attacks
3. 50,000 Portscan Attacks
4. 50,000 Dos Attacks

The time of loading and training were vastly increased, as the loading of csv and storing it in a pandas dataframe was too time and computationaly extensive, as large integers and float values were now stored in float.64 or int.64 datatypes, we explored and experimented with saving the file in .hdf format and experimenting with dask frames instead of pandas frame. As the data was in it's truest form, there was no information loss, hence this greatly improved our performance which was F1 score.

5.1.6.1 Results on Dataset A

Table 5.1 shows the results that we had achieved by training on dataset A.

Models	F1-Score
BERT	64.4
Decision Tree	96.5
RandomForest	96.5
XtraTrees	96.5
Naive-Bayes	35.5
Logistic Regression	50.7
MLP	97.5
AutoEncoders	98.0
VAE	98.95

Table 5.1: Results that we achieved by training on dataset A

5.1.7 Dataset B

What differs our dataset B from the previous data is that each feature value was normalised and scaled to 255, the datatype of each feature could now be converted to unsigned int; this came with it's own drawbacks and positive feedbacks.

Dataset A is the dataset which contains a total of 800,000 samples:

1. 10,0000 Benign Streams
2. 100,000 DDos Attacks
3. 100,000 Portscan Attacks
4. 100,000 Dos Attacks

As the data was not in it's truest form, there was great information loss, hence this greatly declined our accuracy measure, which was our F1 score. The time of loading and training dropped down vastly, the whole dataframe which was once of gigabytes was now in megabytes; this saved us training and computational costs.

5.1.7.1 Results on Dataset B

5.2 shows the results that we had achieved by training on dataset B.

Models	F1-Score
Decision Tree	77.5
RandomForest	81.5
XtraTrees	80.5
MLP	81.5
AutoEncoders	84.0
VAE	87.95

Table 5.2: Results that we achieved by training on Dataset B

Chapter 6

Discussion

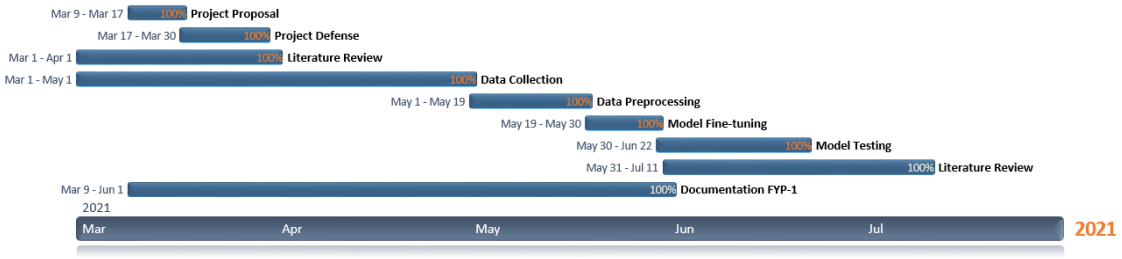
We started off this project with the basic idea of protecting internet-of-things devices from malicious applications(Mirai, Gafgyt etc.) as since the advent of 5g networks has given birth to a rise of malicious IoT devices. All the literature we reviewed sparked an interest inside us to understand and pursue the domain of Computer Networks. As, Computer Networks is a pretty vast field, and there's a lot of minute details to know in order to truly take advantage of machine or to protect a machine from an adversary, we had to analyse and remove the layers of abstraction; which is how several machines communicate via the network. In order to take advantage of Artificial Intelligence or Deep Learning in our case, we needed to understand the intricacies of how a malicious stream varies from benign stream. The intricacies were of extreme importance for us as a neural network greatly utilizes the minute patterns, details and intricacies of data.

we had to keenly construct and distribute our deliverance of work in nature to the time we had. As our project was distributed into two parts; FYP-1 and FYP-2, we had to act accordingly. Due to extensive literature research, we managed to learn what was needed to be done to achieve our task. We then carefully fomalized how to achieve what task, and when to achieve that task.

6.1 Work Breakdown for FYP-1

In Final Year Project Part 1 we have worked on collecting data that was most difficult phase because usually its not that easy in Network Security domain and also availability of public/ free dataset is rear but we manage to collect data of almost 2 terabytes, one assuming we are having sufficient data, we step towards visualizing and understanding of those dataset, once done with it, we moved forward towards Data Pre-processing. We applied different Normalization techniques on data to thoroughly understanding the behaviour of our data. Once we had done that, we take a comprehensive Literature review for appropriate model selection, although we had one model in our mind to fine tune.

We started our training phase by training and testing of BERT, as we know BERT is a state of art model, developed and used by google recently. So, the issues we came across were computational resources required for training(fine-tuning) of a model. BERT base model have 12 layers (transformer blocks), 12 attention heads, and 110 million parameters. It took us thorough literature review and research to completely understand Transformer architecture, BERT and its hidden layers and only layer that can be fine-tune. Once we have fine-tuned the BERT model we started testing phase that took us almost a week.

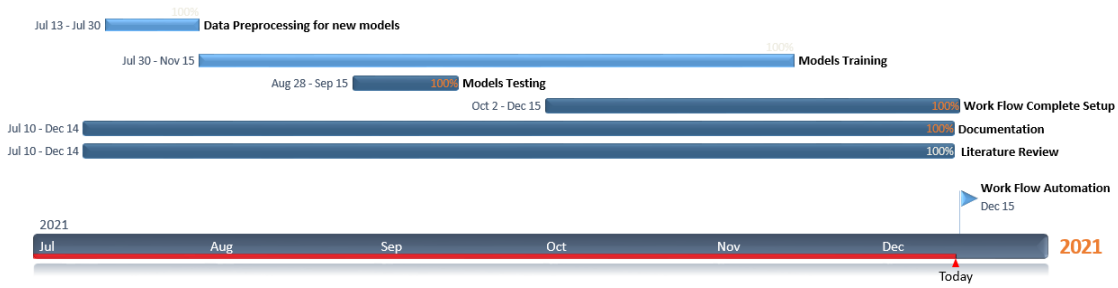


Work Breakdown for FYP-1

6.2 Work Breakdown for FYP-2

In FYP-2 we move forward to train more models to achieve accuracy and to get good results that was not provided by BERT due to resources limitations. We trained and tested several machine learning and deep learning models, which were DecisionTree, Random-Dorest, ExtraTree, AuroEncoders, Variational AutoEncoders, GANs. With training and testing of these model our next phase start to put together our workflow and automate it using python scripts.

Once we had trained and tested our model, now we had to devise such a mechanism that it captures network traffic from a live network, and dump into a file , than change that file from pcap to csv and apply preprocessing and normalization techniques, load the weights of trained model in a file and predict the captured file, once model is integrated and produces the appropriate results, if a model predicts the activity as malicious, dump the information in a log file with a detailed info of network stream. We have almost completed this phase in 3 months.



Work Breakdown for FYP-2

Chapter 7

Conclusions and Future Work

We used deep neural networks to design an anomaly detection system. The system can detect, classify, and generate alarms via log file for abnormal traffic in the network. We have concluded the approach we have carried out can be used in a variety of domains of cybersecurity areas, To detect Malicious Network Activity we take features such as Arriving Packet Length, size of payload, time of arrival in consideration, however we have removed misleading features such as source IP to prevent it from being effected by fraud IP or fraud MAC addresses.

We have experimented to deep neural language model such as State-of-the-art Bidirectional Encoder Representation of Transformers, but were unable to attain optimum results due to lack of computational resources as a big a model as bert can truly reach it's maximum potential if it were provided the required computational resources. We have explored the domain of Generative Models such as AutoEncoders and Variational autoencoders and are pleased to claim that we have achieved pretty as well.

In the modern era, to be truly safe on the internet is no less than a never ending challenge, as with time progressing, as technology advances; the ones with malicious intents also evolve their ways to take advantage. Hence to be truly secure on the internet, an individual must take several precautionary measures to mantain cyber-privacy.

Appendix A

Data Preprocessing

Source Code

4.1 Excerpt of Parser Class 36
4.2 Excerpt of Preprocessor Class 37
A.1 Parser Class 69
A.2 Preprocessor Class 72
B.1 MLP Model 79
B.2 AutoEncoder Model 80

A.1 Parser Class

```
from lxml import etree

# xml parser object
doc = etree.iterparse('2012\TestbedSunJun13Flows.xml',  events=('
    start',), tag='TestbedSunJun13Flows' )
print(doc)

```

```
# Appending header row into csv file
filename =  etree.iterparse('2012\TestbedSunJun13Flows.xml',
    events=('start',), tag='TestbedSunJun13Flows' )
class Parser:
  def __init__(self):
    self.doc = etree.iterparse(filename,  events=('start',), tag='
    TestbedSunJun13Flows' )
    self.csv = []
    with open('flowSunJun115.csv', 'a') as f: #Write CSV File
        header =
        'appName,source,protocolName,sourcePort,destination,
    destinationPort,direction,totalSourceBytes,
    totalDestinationBytes,totalDestinationPackets,
    totalSourcePackets,sourcePayloadAsBase64,sourcePayloadAsUTF,
    destinationPayloadAsBase64,destinationPayloadAsUTF,
    sourceTCPFlagsDescription, destinationTCPFlagsDescription,
    startDateTime,stopDateTime,Tag'

        f.writelines(header+"\n")

  def write_as_csv(self,row):
      """
          write dictionary into csv
      """
      r = ' , '.join(list(row.values()))

      with open('flowSunJun115.csv', 'a') as f:
          f.writelines(r+"\n")
      print(r)


  # csv = []
  def extract_row(self,dom):
      tags = {
            'appName' : '',
            'source': '',
            'protocolName': '',
```

```
36              'sourcePort': '',
37              'destination': '',
38              'destinationPort': '',
39              'direction': '',
40              'totalSourceBytes': '',
41              'totalDestinationBytes': '',
42              'totalDestinationPackets': '',
43              'totalSourcePackets': '',
44              'sourcePayloadAsBase64': '',
45              'sourcePayloadAsUTF': '',
46              'destinationPayloadAsBase64': '',
47              'destinationPayloadAsUTF': '',
48              'sourceTCPFlagsDescription': '',
49              'destinationTCPFlagsDescription': '',
50              'startDateTime': '',
51              'stopDateTime': '',
52              'Tag': ''
53      }
54
55        # for each object in xml
56        # assign each value to it's
57        # tag in above defined tag's
58        # dictionary
59        for d in dom:
60            if not d.text or d.text == None:
61            tags[d.tag] = '0'
62
63        elif ',' in d.text:
64            tags[d.tag] = ' '.join(d.text.split(','))
65        else:
66            tags[d.tag] = d.text
67
68        write_as_csv(tags)
69        return tags
70
71
72  def extract_children(self, self.doc):
```

```
  for event, element in self.doc:
    children = element.getchildren()
    extract_row(children)
#   print(children)

#   <-------------- END OF PARSER --------------->
```

Source Code A.1: Parser Class

A.2 Preprocessor Class

```

import threading

'''
    Preprocessor class:
    this is the class which will be used to manipulate our data.
    -> self.df; is the member; in which the dataframe will be loaded.
    -> Preprocessor: is used to clean & pre-process our data.

'''

class Preprocessor:

    def __init__(self, csv_path='/content/drive/MyDrive/MNAD-FYP/FYP-1/flowSunJun115.csv'):
        #Reading the csv file pn creation & loading it in dataframe.
        self.df = pd.read_csv(csv_path)

    def get_df(self):
        #Returns DataFrame
        return self.df

    def drop_outlier(self, outliers):
```

```
        #Will be used to Remove repeated values & outliers;
        # [* helper function] - for outlier_remover

        print("Removing indexes : ",outliers )

        #inplace = True; Manipulate inside the current df,
        self.df.drop(outliers, inplace=True)
        print("Removed outliers !")                    #^ don't make a new df, ^
        print(self.df.shape)

    def rem_space(self, valToRemove1,valToRemove2):

        """
            REMOVE_space or whatever, passed in valToRemove1 & valToRemove2
            Will get List of indexes of where valToRemove1 &
            valToRemove2 are situated.
        """

        valToRemove1 = str(valToRemove1)
        valToRemove2 = str(valToRemove2)

        df = self.get_df()
        outlier_indexes = df.loc[df['Tag'].isin([valToRemove1, valToRemove2])].index #Extract Rows w valToRemove1 & valToRemove2 in em

        print(type(outlier_indexes[0]))
        print((outlier_indexes[0]))

        return outlier_indexes       #Returns a list of indexes to remove

    def shape(self):

```

```
        """
            Returns Shape of the dataframe
        """

        return self.df.shape

    def outlier_remover(self):

        """
            Extracts indexes of outlier rows and
            sends them to be removed.
        """

        outlier_list = self.rem_space(' ', '0')
        print("Outliers Count : ",len(outlier_list))

        self.drop_outlier(outlier_list)
        print("[*] Done !\nNew shape : ",self.df.shape)

    def attackNormalVisualize(self):

        """
            func will display barplot to visual attack and Normal
    data ratio.
        """

        %matplotlib inline
        tag = self.df['Tag']
        tag.value_counts().plot(kind='bar')# White Spaces in Tag
    Label & NaN; as shown in barplot

        print(tag.value_counts()[0])
        plt.ylabel('Frequency')
        plt.xlabel('Labels/Tags of Network Packets');
        plt.title("captured network packets")
        plt.show()

```

```
    def tagZeroRemoval(self):

        '''
            Will drop rows with false 'Tag' label;
            which can be spaces or null values
        '''
        zeroOutliers = self.df.loc[self.df['Tag'].isin(['0 ', ' 0'
    ,' ' ])].index
        print(zeroOutliers)

        self.df = self.df.drop(zeroOutliers)

    def attackProbability(self):

        '''
           Attack frequency / (AttackFrequency + NormalFrequency)
        '''

        df_tag = self.df['Tag']
        df_tag.value_counts()

        Attack_count = df_tag.value_counts()[1]
        Benign_count = df_tag.value_counts()[0]

        TotalOutcomes = (Attack_count + Benign_count)
        Attack_proba = ((Attack_count / (TotalOutcomes)) )

        print(Attack_count ,'/', Attack_count + Benign_count, ',
    Attack_count:',Attack_count,'Benign_count:',Benign_count)
        print(Attack_proba, 'Probability of Attack')
#         return Attack_proba

    def fill_NA(self):

        """
            Replace coloumn values with a null value
```

```
            if the column value is N/A. We can change
            the columns later, on whicb the rows have
            to be replaced .
        """

        self.df = self.df.fillna(0)
        self.df['sourceTCPFlagsDescription'] = df['
sourceTCPFlagsDescription'].replace('N/A', 0)
        self.df['destinationTCPFlagsDescription'] = df['
destinationTCPFlagsDescription'].replace('N/A', 0)

        #returns dataframe with filled NAs
        return self.df

    def getTagValues(self):

        """
            Return tag label counts
        """

        TagValues = self.df['Tag']
        return TagValues.value_counts()

    def scatter_plot(self):

        """
            Visulize scatter plot
        """

        plt.figure(figsize=(10, 5))

        x =  range(self.df['Tag'].size)
        y =  self.df['destinationPayloadAsUTF']

        plt.scatter(x, y, marker='x', s=3.35)
        plt.title("Relationship between Malicious & Benign
Activity")
```

```
        plt.xlabel("Packet Arrival Order")
        plt.ylabel("Labels")

        plt.show()


    def dropCertainColumns(self):

        """
           Remove some more unnecessary data.
        """

        self.df = self.df.drop(columns=
            ['totalSourcePackets',
             'totalDestinationPackets',
             'startDateTime',
             'stopDateTime'])

        return self.df
```

Source Code A.2: Preprocessor Class

Appendix B

Models

B.1 Muti-Layer Perceptron

```

# Set the input shape
input_shape = (feature_vector_length,)
print(f'Feature shape: {input_shape}')

# Create the model
model = Sequential()

model.add(Input(shape=(input_shape )))

model.add(Dense(28, activation='relu'))
model.add(Dropout(0.2))

model.add(Dense(14, activation='relu'))
model.add(Dropout(0.2))

model.add(Dense(7, activation='relu'))
model.add(Dropout(0.2))

model.add(Dense(num_classes, activation='softmax')
```

Source Code B.1: MLP Model

B.2 AutoEncoder Model

```
encoder = Sequential()


nb_epoch = 200
batch_size = 32

learning_rate = 1e-3
input_layer = Input(shape=(55, ))

def encoder(input_layer):

  encoder1 = Dense(32, activation="relu")(input_layer) #32 16 8 ,
   activation="relu"
  encoder1 = Dropout(0.2)(encoder1)

  encoder2 = Dense(16, activation="relu")(encoder1)
  encoder2 = Dropout(0.2)(encoder2)

  encoder3 = Dense(8, activation="relu")(encoder2)
  encoder3 = Dropout(0.2)(encoder3)

  encoder4 = Dense(4)(encoder3)

  return encoder4

def decoder(encoder4):
  decoder1 = Dense(8, activation="relu")(encoder4)
  decoder1 = Dropout(0.2)(decoder1)
  # decoder1 = BatchNormalization()(decoder1)

  decoder2 = Dense(16, activation="relu")(decoder1)
  decoder2 = Dropout(0.2)(decoder2)
  # decoder2 = BatchNormalization()(decoder2)
```

```
34
35    decoder3 = Dense(32, activation="relu")(decoder2)
36    decoder3 = Dropout(0.2)(decoder3)
37
38    decoder4 = Dense(55)(decoder3) #no activation
39
40    return decoder4
41
42 autoencoder = Model(input_layer, decoder(encoder(input_layer)))
43 autoencoder.get_weights()[0][1]
44
45 autoencoder.compile(optimizer='adam', loss='mse', metrics=['mse'])
46 autoencoder.summary()
47
48 autoencoder_train = autoencoder.fit(x_train,y_train,
49                                      batch_size=batch_size,
50                                      epochs=200,
51                                      verbose=1,
52                                      validation_data=(x_test,
     y_test))
53
54
55 nb_epoch = 100
56 batch_size = 64
57
58 learning_rate = 1e-3
59 input_layer = Input(shape=(55, ))
60
61 def encoder(input_layer):
62
63    encoder1 = Dense(32, activation="relu")(input_layer) #32 16 8 ,
      activation="relu"
64    encoder1 = Dropout(0.2)(encoder1)
65
66
67    encoder2 = Dense(16, activation="relu")(encoder1)
68    encoder2 = Dropout(0.2)(encoder2)
```

```
69
70    encoder3 = Dense(8, activation="relu")(encoder2)
71    encoder3 = Dropout(0.2)(encoder3)
72
73    encoder4 = Dense(4)(encoder3)
74    return encoder4
75
76 def fc(enco):
77     flat = Flatten()(enco)
78     den = Dense(64, activation='relu')(flat)
79     out = Dense(4, activation='softmax')(den)
80     return out
81
82 encode = encoder(input_layer)
83 full_model = Model(input_layer,fc(encode))
84
85 for l1,l2 in zip(full_model.layers[:8],autoencoder.layers[0:8]):
86     l1.set_weights(l2.get_weights())
87
88 for layer in full_model.layers[0:10]:
89     layer.trainable = False
```

Source Code B.2: AutoEncoder Model

Bibliography

[1] Splitcap. https://www.netresec.com/?page=SplitCap, 2010.

[2] Intrusion detection evaluation dataset (CIC-IDS2012) . https://www.unb.ca/cic/datasets/ids.html, 2012.

[3] CSE-CIC-IDS2018 on AWS. https://www.unb.ca/cic/datasets/ids-2018.html, 2018.

[4] Intrusion detection evaluation dataset (CIC-IDS2017). https://www.unb.ca/cic/datasets/ids-2017.html, 2018.

[5] PRC Flowmeter v0.2.0. https://github.com/prc-hsv/flowmeter/, 2021.

[6] M. Guizani A. M. N. Zaza, S. K. Kharroub and K. Abualsaud. Lightweight iot malware detection solution using cnn classification. *arXiv [cs.CR]*, 2020.

[7] Aldribi A. Alshammari, A. Apply machine learning techniques to detect malicious network traffic in cloud computing. *Journal of Big Data*, pages 1–24, 06 2019.

[8] Niki Parmar Jakob Uszkoreit Llion Jones Aidan N. Gomez Ashish Vaswani, Noam Shazeer and Łukasz Kaiser. Attention is all you need. *arXiv:1706.03762v5 [cs.CL]*, 2017.

[9] and Shawn E. Davis Benjamin J. Radford, Bartley D. Richardson. Sequence aggregation rules for anomaly detection in computer network traffic. *arXiv:1805.03735v2 [cs.CR]]*, 2018.

[10] Satchidananda Dehuri Bhabani Shankar Prasad Mishra, Himansu Das and Alok Kumar Jagadev. Cloud Computing for Optimization and Foundations Applications and Challenges. *Part of the Studies in Big Data book series*, 39:1–463, 2018.

[11] Jason Brownlee. Multi-layer perceptron neural networks. https://machinelearningmastery.com/neural-networks-crash-course/, 2016.

[12] Ferhat Ozgur Catak and Ahmet Mustacoglu. Distributed denial of service attack detection using autoencoder and deep neural networks. *Journal of Intelligent Fuzzy Systems*, 37:1–11, 07 2021.

[13] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805, 2018.

[14] Divya Bansal Ekta Gandotra and Sanjeev Sofat. Malware analysis and classification: A survey. *Journal of Information Security*, 2014.

[15] Purvi Prajapati Harsh H. Patel. Study and analysis of decision tree based classification algorithms. *International Journal of Computer Sciences and Engineering*, 2018.

[16] Arash Habibi Lashkari Iman Sharafaldin and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. *4th International Conference on Information Systems Security and Privacy*, 2018.

[17] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. Albert: A lite bert for self-supervised learning of language representations, 2020.

[18] Xuewen Zeng Luchao Han and Lei Song. A novel transfer learning based on albert for malicious network traffic classification. *ICIC International ©2020 ISSN 1349-4198*, 2020.

[19] Xuewen Zeng Luchao Han and Lei Song. A novel transfer learning based on albert for malicious network traffic classification. *ICIC International©2020 ISSN 1349-4198]*, 2020.

[20] Rajvardhan Oak, Min Du, David Yan, Harshvardhan Takawale, and Idan Amit. Malware detection on highly imbalanced data through sequence modeling. In *Proceedings of the 12th ACM Workshop on Artificial Intelligence and Security*, AISec'19, page 37–48, New York, NY, USA, 2019. Association for Computing Machinery.

[21] Jiri Havelka Paul Prasse, Tomas Pevny. Malware detection by analyzing network traffic with neural networks. *IEEE Symposium on Security and Privacy workshop*, 2017.

[22] Mahbod Tavallaee, Ebrahim Bagheri, Wei Lu, and Ali A. Ghorbani. A detailed analysis of the kdd cup 99 data set. In *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications*, pages 1–6, 2009.

[23] Wireshark. Dumpcap 3.6.0. www.wireshark.org/docs/man-pages/dumpcap.html, 2021.

[24] Dabao Wei Yirui Wu and Jun Feng. Network attacks detection methods based on deep learning techniques: A survey. *Hindawi Security and Communication Networks*, 2020.

[25] Sebastian Goodman Kevin Gimpel Piyush Sharma Zhenzhong Lan, Mingda Chen and Radu Soricut. Albert: A lite bert for self-supervised learning of language representations. *arXiv:1909.11942v6 [cs.CL]*, 2020.